

Wszystkie_zadania_bez_notatek

Lab 1

Zadanie 1

Sprawdź, czy suma dwóch liczb zmiennoprzecinkowych (zarówno w pojedynczej jak i podwójnej precyzji) jest zawsze równa oczekiwanej matematycznie wartości. Wyjaśnij, dlaczego odpowiedź może być błędna przy bezpośrednim porównaniu liczb zmiennoprzecinkowych.

PYTHON

```
import numpy as np

def suma_pojedyncza_precyzja(a, b):
    a32 = np.float32(a)
    b32 = np.float32(b)
    return np.float32(a32 + b32)

poj = suma_pojedyncza_precyzja(0.1, 0.2)

print(poj)

def suma_podwojna_precyzja(a: float, b: float) -> float:
    return a + b

pod = suma_podwojna_precyzja(0.1, 0.2)

print(pod)

print("Czy równe?: ", poj == pod)
```

```
0.3
0.30000000000000004
Czy równe?: False
```

Zadanie 2

Sprawdź, co się stanie, gdy dodasz bardzo dużą liczbę do bardzo małej, a następnie odejmiesz dużą liczbę od wyniku.

PYTHON

```
a = 10000000000000000
b = 0.0000000000000001

wynik = (a + b) - a

print(wynik)
```

0.0

Zadanie 3

Wyświetl liczbę zmiennoprzecinkową jako float i double z precyzją do 20 miejsc po przecinku.

PYTHON

```
import numpy as np

liczba = 0.1

# float - pojedyncza precyzja
liczba_f = np.float32(liczba)

# double - podwójna precyzja
liczba_d = float(liczba)

print("float: ", format(liczba_f, ".20f"))
print("double: ", format(liczba_d, ".20f"))
```

```
float:  0.10000000149011611938
double: 0.10000000000000000555
```

Zadanie 4

Oblicz $0,3 \cdot 3 + 0,1$ i porównaj wynik z jego wartościami zaokrąglonymi do dołu i do góry (floor i ceil).

PYTHON

```
import math

wynik = 0.3 * 3 + 0.1

print("Wynik zwykły: ", wynik)
print("Wynik zaokrąglony w dół: ", math.floor(wynik))
print("Wynik zaokrąglony w górę: ", math.ceil(wynik))
```

Wynik zwykły: 0.9999999999999999

Wynik zaokrąglony w dół: 0

Wynik zaokrąglony w górę: 1

Zadanie 5

Oblicz różnicę między 1,0000001 i 1,0000000 oraz między 1,0000002 i 1,0000001. Wyjaśnij, dlaczego wyniki mogą się różnić od teoretycznej różnicy.

PYTHON

```
wynik1 = 1.0000001 - 1.0000000
wynik2 = 1.0000002 - 1.0000001

print("wynik1: ", wynik1)
print("wynik2: ", wynik2)
```

```
wynik1: 1.0000000005838672e-07
wynik2: 9.999999983634211e-08
```

Zadanie 6

Podziel liczbę 1,0 przez 0,0 i liczbę 0,0 przez 0,0. Sprawdź, co zwrócą te operacje.

PYTHON

```
# wynik1 = 1.0 / 0.0
wynik1 = np.divide(1.0, 0.0)
# wynik2 = 0.0 / 0.0
wynik2 = np.divide(0.0, 0.0)

print("wynik dzielenia pierwszego: ", wynik1)
print("wynik dzielenia drugiego: ", wynik2)
```

Wynik z NumPy:

```
zadania.py:80: RuntimeWarning: divide by zero encountered in divide
  wynik1 = np.divide(1.0, 0.0)
zadania.py:82: RuntimeWarning: invalid value encountered in divide
  wynik2 = np.divide(0.0, 0.0)
wynik dzielenia pierwszego: inf
wynik dzielenia drugiego: nan
```

Wynik z czystego pythona:

```
Traceback (most recent call last):
  File "zadania.py", line 79, in <module>
    wynik1 = 1.0 / 0.0
              ~~~~^~~~~
ZeroDivisionError: float division by zero
```

```
-----

Traceback (most recent call last):
  File "zadania.py", line 80, in <module>
    wynik2 = 0.0 / 0.0
              ~~~~^~~~~
ZeroDivisionError: float division by zero
```

Zadanie 7

Oblicz maszynowy epsilon dla typów float i double i porównaj wyniki. Wyjaśnij, czym jest maszynowy epsilon i jak wpływa na dokładność obliczeń komputerowych.

```

import numpy as np

def epsilon_float32():
    eps = np.float32(1.0)

    while np.float32(1.0) + eps / np.float32(2.0) > np.float32(1.0):
        eps = eps / np.float32(2.0)

    return eps

def epsilon_float64():
    eps = 1.0

    while 1.0 + eps / 2.0 > 1.0:
        eps = eps / 2.0

    return eps

eps32 = epsilon_float32()
eps64 = epsilon_float64()

u32 = eps32 / np.float32(2.0)
u64 = eps64 / 2.0

print("Epsilon float32:", eps32)
print("Unit roundoff float32:", u32)

print("Epsilon float64 / double:", eps64)
print("Unit roundoff float64:", u64)

print("Porównanie:")
print("float32 ma dokładność około 7 cyfr dziesiętnych")
print("float64 / double ma dokładność około 16 cyfr dziesiętnych")

```

W programie obliczono wartość epsilon jako najmniejszą liczbę dodatnią, dla której $1 + \text{epsilon} > 1$. W literaturze czasem przez maszynowy epsilon rozumie się tę wartość, a czasem połowę tej wartości, czyli tzw. unit roundoff u , zgodny ze wzorem ze slajdu. Dlatego dla zaokrąglania do najbliższej liczby maszynowej przyjmuje się $u \approx /2$.

Epsilon float32: 1.1920929e-07
Unit roundoff float32: 5.9604645e-08
Epsilon float64 / double: 2.220446049250313e-16
Unit roundoff float64: 1.1102230246251565e-16
Porównanie:
float32 ma dokładność około 7 cyfr dziesiętnych
float64 / double ma dokładność około 16 cyfr dziesiętnych

Zadanie 8

Sumuj liczbę 0,0001 w pętli 1.000.000 razy i porównaj wynik z wynikiem uzyskanym przez mnożenie 1.000.000 przez 0,0001. Wyjaśnij, dlaczego mogą wystąpić różnice.

PYTHON

```
# liczba iteracji
n = 1_000_000
wartosc = 0.0001

# Sumowanie w pętli
suma_petla = 0.0
for _ in range(n):
    suma_petla += wartosc

# Mnożenie
suma_mnozenie = n * wartosc

# Różnica
roznica = suma_petla - suma_mnozenie

print("Wynik sumowania w pętli:", suma_petla)
print("Wynik mnożenia:", suma_mnozenie)
print("Różnica:", roznica)
```

Wynik sumowania w pętli: 100.00000000219612
Wynik mnożenia: 100.0
Różnica: 2.1961170659778873e-09

Zadanie 9

Oblicz sumę odwrotności liczb od 1 do 1.000.000 w kolejności rosnącej i malejącej. Porównaj wyniki.

```
n = 1_000_000

# Sumowanie w kolejności rosnącej (od 1 do 1_000_000)
suma_rosnaco = 0.0
for i in range(1, n + 1):
    suma_rosnaco += 1.0 / i

# Sumowanie w kolejności malejącej (od 1_000_000 do 1)
suma_malejaco = 0.0
for i in range(n, 0, -1):
    suma_malejaco += 1.0 / i

# Różnica
roznica = suma_rosnaco - suma_malejaco

print("Suma rosnąco:", suma_rosnaco)
print("Suma malejąco:", suma_malejaco)
print("Różnica:", roznica)
```

```
Suma rosnąco: 14.392726722864989
Suma malejąco: 14.392726722865772
Różnica: -7.833733661755105e-13
```

Zadanie 10

Niech

$$f(x) = \sqrt{x^2 + 1} - 1$$

oraz

$$g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}.$$

Łatwo zauważyć, że $g = f$. Oblicz i porównaj wartości funkcji g i f dla:

$$x = 8^{-1}, 8^{-2}, 8^{-3}, \dots$$

```
import math

def f(x):
    return math.sqrt(x**2 + 1) - 1

def g(x):
    return x**2 / (math.sqrt(x**2 + 1) + 1)

print(f"{'x':>12} {'f(x)':>20} {'g(x)':>20} {'różnica':>20}")

for k in range(1, 11):
    x = 8**(-k)
    fx = f(x)
    gx = g(x)
    print(f"{'x':12.5e} {'fx':20.15e} {'gx':20.15e} {'(fx-gx)':20.15e}")
```

x	f(x)	g(x)	różnica
1.25000e-01	7.782218537318641e-03	7.782218537318706e-03	-6.505213034913027e-17
1.56250e-02	1.220628628286757e-04	1.220628628287590e-04	-8.328027937404281e-17
1.95312e-03	1.907346813823096e-06	1.907346813826566e-06	-3.469446951953614e-18
2.44141e-04	2.980232194360610e-08	2.980232194360612e-08	-1.323488980084844e-23
3.05176e-05	4.656612873077393e-10	4.656612871993190e-10	1.084202172485504e-19
3.81470e-06	7.275957614183426e-12	7.275957614156956e-12	2.646977960169689e-23
4.76837e-07	1.136868377216160e-13	1.136868377216096e-13	6.462348535570529e-27
5.96046e-08	1.776356839400250e-15	1.776356839400249e-15	1.577721810442024e-30
7.45058e-09	0.000000000000000e+00	2.775557561562891e-17	-2.775557561562891e-17
9.31323e-10	0.000000000000000e+00	4.336808689942018e-19	-4.336808689942018e-19

Lab 2

Zadanie 1

Napisz funkcję, która wygeneruje tablicę liczb zmiennoprzecinkowych pojedynczej precyzji reprezentujących elementy ciągu postaci:

$$S_n = \sum_{k=0}^{n-1} a_k = \sum_{k=0}^{n-1} \frac{1}{(k \bmod m + 1)(k \bmod m + 2)},$$

gdzie n i m są potęgami liczby 2 oraz $n > m$.

Uniwersalna funkcja na sprawdzenie czy x jest potęgą y

PYTHON

```
def czy_potega(x, y): # sprawdza, czy x jest potęgą liczby y
    if x < 1 or y <= 1: # potęgi rozważamy dla dodatniego x i podstawy y większej
        od 1
        return False # jeśli warunek nie jest spełniony, zwracamy False

    while x % y == 0: # dopóki x dzieli się przez y bez reszty
        x = x // y # dzielimy x przez y

    return x == 1 # jeśli na końcu zostało 1, to x było potęgą y
```

Przykład użycia

PYTHON

```
print(czy_potega(64, 2)) # True, bo 64 = 2^6
print(czy_potega(81, 3)) # True, bo 81 = 3^4
print(czy_potega(100, 10)) # True, bo 100 = 10^2
print(czy_potega(12, 2)) # False
print(czy_potega(16, 4)) # True, bo 16 = 4^2
```

Rozwiązanie zadania

```
import numpy as np

def czy_potega_dwojki(x):
    return x > 0 and x & (x - 1) == 0

def generuj_tablice(n, m):
    if czy_potega_dwojki(n) and czy_potega_dwojki(m) and n > m:
        tablica = []

        for k in range(n):
            licznik = 1
            mianownik = ((k % m) + 1) * ((k % m) + 2)

            element = licznik / mianownik

            tablica.append(np.float32(element))

        return tablica
    else:
        raise ValueError("n i m muszą być potęgami liczby 2 oraz musi być spełnione n > m")

n = 64
m = 16

tablica = generuj_tablice(n, m)

print("Elementy ciągu:")
for i, wartosc in enumerate(tablica):
    print("a_", i, "=", wartosc)
```

Zadanie 2

Napisz funkcję sumującą elementy tablicy z zadania 1. Sprawdź dokładność otrzymanej sumy.

```

import numpy as np

def czy_potega_dwojki(x):
    return x > 0 and x & (x - 1) == 0

def generuj_tablice(n, m):
    if czy_potega_dwojki(n) and czy_potega_dwojki(m) and n > m:
        tablica = []

        for k in range(n):
            licznik = 1
            mianownik = ((k % m) + 1) * ((k % m) + 2)

            element = licznik / mianownik

            tablica.append(np.float32(element))

        return tablica
    else:
        raise ValueError("n i m muszą być potęgami liczby 2 oraz musi być spełnione n > m")

def sumuj_tablice(tablica):
    suma = np.float32(0.0)

    for element in tablica:
        suma = np.float32(suma + element)

    return suma

n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)

print("Elementy tablicy:")
for i, wartosc in enumerate(tablica):
    print("a_", i, "=", wartosc)

print("\nSuma elementów:", suma)
print("\nSprawdzenie sumy: ", n / (m + 1))

```

Zadanie 3

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Gilla–Møllera. Sprawdź dokładność otrzymanej sumy.

```

import numpy as np

def czy_potega_dwojki(x):
    return x > 0 and x & (x - 1) == 0

def generuj_tablice(n, m):
    if czy_potega_dwojki(n) and czy_potega_dwojki(m) and n > m:
        tablica = []

        for k in range(n):
            licznik = 1
            mianownik = ((k % m) + 1) * ((k % m) + 2)

            element = licznik / mianownik

            tablica.append(np.float32(element))

        return tablica
    else:
        raise ValueError("n i m muszą być potęgami liczby 2 oraz musi być spełnione n > m")

def sumuj_tablice(tablica):
    suma = np.float32(0.0)
    poprawka = np.float32(0.0)

    for element in tablica:
        t = np.float32(suma + element)
        poprawka = np.float32(poprawka + (element - (t - suma)))
        suma = t

    return np.float32(suma + poprawka)

n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)

print("\nSuma:", suma)
print("Sprawdzenie sumy:", n / (m + 1))

```

Zadanie 4

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Kahana. Sprawdź dokładność otrzymanej sumy. Porównaj wyniki wszystkich omówionych metod sumowania.

```

import numpy as np

def czy_potega_dwojki(x):
    return x > 0 and x & (x - 1) == 0

def generuj_tablice(n, m):
    if czy_potega_dwojki(n) and czy_potega_dwojki(m) and n > m:
        tablica = []

        for k in range(n):
            licznik = 1
            mianownik = ((k % m) + 1) * ((k % m) + 2)

            element = licznik / mianownik

            tablica.append(np.float32(element))

        return tablica
    else:
        raise ValueError("n i m muszą być potęgami liczby 2 oraz musi być spełnione n > m")

def sumuj_tablice(tablica):
    suma = np.float32(0.0)

    for element in tablica:
        suma = np.float32(suma + element)

    return suma

def sumuj_tablice_Møller(tablica):
    suma = np.float32(0.0)
    poprawka = np.float32(0.0)

    for element in tablica:
        t = np.float32(suma + element)
        poprawka = np.float32(poprawka + (element - (t-suma)))
        suma = t

    return np.float32(suma + poprawka)

def sumuj_tablice_Kahan(tablica):
    suma = np.float32(0.0)
    c = np.float32(0.0)

    for element in tablica:
        y = np.float32(element - c)
        t = np.float32(suma + y)
        c = np.float32((t-suma) - y)
        suma = t

    return np.float32(suma)

```

```
n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)
suma_m = sumuj_tablice_Møller(tablica)
suma_k = sumuj_tablice_Kahan(tablica)

print("Suma zwykła:", suma)
print("Suma Møller:", suma_m)
print("Suma Kahan:", suma_k)

print("Sprawdzenie dokładności:", n/(m+1))
```

Zadanie 5

Przeprowadź analogiczne działania dla danych w podwójnej precyzji.

```

import numpy as np

def czy_potega_dwojki(x):
    return x > 0 and x & (x - 1) == 0

def generuj_tablice(n, m):
    if czy_potega_dwojki(n) and czy_potega_dwojki(m) and n > m:
        tablica = []

        for k in range(n):
            licznik = 1
            mianownik = ((k % m) + 1) * ((k % m) + 2)

            element = licznik / mianownik

            tablica.append(element)

        return tablica
    else:
        raise ValueError("n i m muszą być potęgami liczby 2 oraz musi być spełnione n > m")

def sumuj_tablice(tablica):
    suma = 0.0

    for element in tablica:
        suma = suma + element

    return suma

def sumuj_tablice_Møller(tablica):
    suma = 0.0
    poprawka = 0.0

    for element in tablica:
        t = suma + element
        poprawka = poprawka + (element - (t-suma))
        suma = t

    return suma + poprawka

def sumuj_tablice_Kahan(tablica):
    suma = 0.0
    c = 0.0

    for element in tablica:
        y = element - c
        t = suma + y
        c = (t-suma) - y
        suma = t

    return suma

```

```
n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)
suma_m = sumuj_tablice_Møller(tablica)
suma_k = sumuj_tablice_Kahan(tablica)

print("Suma zwykła:", suma)
print("Suma Møller:", suma_m)
print("Suma Kahan:", suma_k)

print("Sprawdzenie dokładności:", n/(m+1))
```

Lab 3

Zadanie 1

Napisz program, który obliczy normę: euklidesową, Manhattan, maximum dla - wymiarowego wektora.


```
import math

def normy_wektora(wektor):

    # norma euklidesowa
    suma_kwadratow = 0
    for x in wektor:
        suma_kwadratow += x**2
    norma_euklidesowa = math.sqrt(suma_kwadratow)

    # norma Manhattan
    norma_manhattan = 0
    for x in wektor:
        norma_manhattan += abs(x)

    # norma maximum
    # norma_max = max(abs(x) for x in wektor)
    norma_max = 0

    for x in wektor:
        wartosc_bezwzgledna = abs(x)

        if wartosc_bezwzgledna > norma_max:
            norma_max = wartosc_bezwzgledna

    return norma_euklidesowa, norma_manhattan, norma_max

wektor = [3, 4, 5]

euklidesowa, manhattan, maksimum = normy_wektora(wektor)

print("Norma euklidesowa:", euklidesowa)
print("Norma Manhattan:", manhattan)
print("Norma maximum:", maksimum)
```

```
Norma euklidesowa: 7.0710678118654755
Norma Manhattan: 12
Norma maximum: 5
```

Zadanie 2

Napisz program, który będzie wyliczał odległość pomiędzy dwoma punktami przestrzeni dwuwymiarowej w metrykach: euklidesowej, Manhattan, rzece i kolejowej.

```

import math

def odleglosci(P, Q):
    p1, p2 = P
    q1, q2 = Q

    # metryka euklidesowa
    euklidesowa = math.sqrt(math.pow((q1 - p1), 2) + math.pow((q2 - p2), 2))

    #metryka Manhattan
    manhattan = abs(p1 - q1) + abs(p2 - q2)

    #metryka rzeka
    if p1 == q1:
        rzeka = abs(p2 - q2)
    else:
        rzeka = abs(p1 - q1) + abs(p2) + abs(q2)

    # metryka kolejowa
    det = p1 * q2 - p2 * q1
    if det == 0:
        kolejowa = math.sqrt(math.pow((q1 - p1), 2) + math.pow((q2 - p2), 2))
    else:
        kolejowa = math.sqrt(math.pow((0 - p1), 2) + math.pow((0 - p2), 2)) +
        math.sqrt(math.pow((0 - q1), 2) + math.pow((0 - q2), 2))

    return euklidesowa, manhattan, rzeka, kolejowa

punktP = (2, 3)
punktQ = (5, 7)

euklidesowa, manhattan, rzeka, kolejowa = odleglosci(punktP, punktQ)

print("Metryka euklidesowa:", euklidesowa)
print("Metryka Manhattan:", manhattan)
print("Metryka rzeka:", rzeka)
print("Metryka kolejowa/centrum:", kolejowa)

```

```

Metryka euklidesowa: 5.0
Metryka Manhattan: 7
Metryka rzeka: 13
Metryka kolejowa/centrum: 12.207876542506616

```

Lub

```
import math

def metryka_euklidesowa(P, Q):
    p1, p2 = P
    q1, q2 = Q

    return math.sqrt(math.pow(q1 - p1, 2) + math.pow(q2 - p2, 2))

def metryka_manhattan(P, Q):
    p1, p2 = P
    q1, q2 = Q

    return abs(p1 - q1) + abs(p2 - q2)

def metryka_rzeka(P, Q):
    p1, p2 = P
    q1, q2 = Q

    if abs(p1 - q1) < 1e-12:
        return abs(p2 - q2)
    else:
        return abs(p1 - q1) + abs(p2) + abs(q2)

def metryka_kolejowa(P, Q):
    p1, p2 = P
    q1, q2 = Q

    det = p1 * q2 - p2 * q1

    if abs(det) < 1e-12:
        return math.sqrt(math.pow(q1 - p1, 2) + math.pow(q2 - p2, 2))
    else:
        odleglosc_P_od_centrum = math.sqrt(math.pow(p1, 2) + math.pow(p2, 2))
        odleglosc_Q_od_centrum = math.sqrt(math.pow(q1, 2) + math.pow(q2, 2))

        return odleglosc_P_od_centrum + odleglosc_Q_od_centrum

punktP = (2.5, 3.1)
punktQ = (5.2, 7.4)

euklidesowa = metryka_euklidesowa(punktP, punktQ)
manhattan = metryka_manhattan(punktP, punktQ)
rzeka = metryka_rzeka(punktP, punktQ)
kolejowa = metryka_kolejowa(punktP, punktQ)

print("Metryka euklidesowa:", euklidesowa)
print("Metryka Manhattan:", manhattan)
```

```
print("Metryka rzeka:", rzeka)
print("Metryka kolejowa/centrum:", kolejowa)
```

Zadanie 3

Napisz program, który obliczy normę: Frobeniusa, Manhattan, maximum dla - wymiarowej macierzy.

PYTHON

```
import math

def normy_macierzy(macierz):
    wiersze = len(macierz)
    kolumny = len(macierz[0])

    # norma Frobeniusa
    suma_kwadratow = 0
    for i in range(wiersze):
        for j in range(kolumny):
            suma_kwadratow += math.pow(macierz[i][j], 2)
    Frobeniusa = math.sqrt(suma_kwadratow)

    # norma Manhattan
    suma_modulow = 0
    for i in range(wiersze):
        for j in range(kolumny):
            suma_modulow += abs(macierz[i][j])
    Manhattan = suma_modulow

    # norma maksimum
    maksimum = 0
    for i in range(wiersze):
        for j in range(kolumny):
            if abs(macierz[i][j]) > maksimum:
                maksimum = abs(macierz[i][j])

    return Frobeniusa, Manhattan, maksimum

macierz = [
    [1, -2, 3],
    [4, 5, -6]
]

Frobeniusa, Manhattan, maksimum = normy_macierzy(macierz)

print("Norma Frobeniusa:", Frobeniusa)
print("Norma Manhattan:", Manhattan)
print("Norma maksimum:", maksimum)
```

Norma Frobeniusa: 9.539392014169456

Norma Manhattan: 21

Norma maksimum: 6

Każda funkcja oddzielnie

```
import math

def norma_Frobeniusa(macierz):
    wiersze = len(macierz)
    kolumny = len(macierz[0])

    suma_kwadratow = 0

    for i in range(wiersze):
        for j in range(kolumny):
            suma_kwadratow += math.pow(macierz[i][j], 2)

    return math.sqrt(suma_kwadratow)

def norma_Manhattan(macierz):
    wiersze = len(macierz)
    kolumny = len(macierz[0])

    suma_modulow = 0

    for i in range(wiersze):
        for j in range(kolumny):
            suma_modulow += abs(macierz[i][j])

    return suma_modulow

def norma_maksimum(macierz):
    wiersze = len(macierz)
    kolumny = len(macierz[0])

    maksimum = 0

    for i in range(wiersze):
        for j in range(kolumny):
            if abs(macierz[i][j]) > maksimum:
                maksimum = abs(macierz[i][j])

    return maksimum

macierz = [
    [1, -2, 3],
    [4, 5, -6]
]

Frobeniusa = norma_Frobeniusa(macierz)
Manhattan = norma_Manhattan(macierz)
maksimum = norma_maksimum(macierz)

print("Norma Frobeniusa:", Frobeniusa)
```

```
print("Norma Manhattan:", Manhattan)
print("Norma maksimum:", maksimum)
```

Zadanie 4

Napisz program, który wykona mnożenie dwóch macierzy. Kiedy działanie takie nie może zostać przeprowadzone? Sprawdź czy mnożenie macierzy jest przemienne lub łączne?

```

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz1 = [
    [1, 2],
    [3, 4]
]

macierz2 = [
    [5, 6],
    [7, 8]
]

wynik = mnozenie_macierzy(macierz1, macierz2)

print("Wynik mnożenia:")
for wiersz in wynik:
    print(wiersz)

macierzAB = mnozenie_macierzy(macierz1, macierz2)
macierzBA = mnozenie_macierzy(macierz2, macierz1)
# czy przemienne => nie jest
print("Czy AB == BA", macierzAB == macierzBA)

C = [
    [2, 0],
    [0, 2]
]

macierzAB = mnozenie_macierzy(macierz1, macierz2)
lewa = mnozenie_macierzy(macierzAB, C)

macierzBC = mnozenie_macierzy(macierz2, C)

```



```
prawa = mnozenie_macierzy(macierz1, macierzBC)
```

```
# czy łączne => jest łączne
```

```
print("Czy (AB)C == A(BC)", lewa == prawa)
```

Wynik mnożenia:

[19, 22]

[43, 50]

Czy AB == BA False

Czy (AB)C = A(BC) True

Zadanie 5

Zaprojektuj i utwórz klasę dla macierzy umożliwiającą tworzenie, wypisywanie i wykonywanie działań: mnożenie przez stałą, dodawanie, mnożenie.

```

# definicja klasy Macierz
class Macierz:
    # funkcja uruchamiana przy tworzeniu nowego obiektu klasy
    def __init__(self, dane):
        # sprawdzamy, czy macierz nie jest pusta
        if len(dane) == 0:
            raise ValueError("Macierz nie może być pusta")

        # sprawdzamy, czy pierwszy wiersz nie jest pusty
        if len(dane[0]) == 0:
            raise ValueError("Macierz musi mieć co najmniej jedną kolumnę")

        # zapamiętujemy liczbę kolumn z pierwszego wiersza
        liczba_kolumn = len(dane[0])

        # sprawdzamy, czy wszystkie wiersze mają tyle samo kolumn
        for wiersz in dane:
            if len(wiersz) != liczba_kolumn:
                raise ValueError("Wszystkie wiersze macierzy muszą mieć tę samą
długość")

        # zapisanie danych macierzy wewnątrz obiektu
        self.dane = dane

    # funkcja do wypisywania macierzy
    def wypisz(self):
        # przejście po wszystkich wierszach macierzy
        for wiersz in self.dane:
            # wypisanie jednego wiersza
            print(wiersz)

    # funkcja mnożąca macierz przez liczbę
    def mnozenie_przez_stala(self, stala):
        # pusta lista na wynik
        wynik = []

        # przejście po wszystkich wierszach macierzy
        for wiersz in self.dane:
            # nowy wiersz wynikowy
            nowy_wiersz = []

            # przejście po wszystkich elementach w danym wierszu
            for element in wiersz:
                # dodanie do nowego wiersza elementu pomnożonego przez stałą
                nowy_wiersz.append(element * stala)

            # dodanie gotowego wiersza do macierzy wynikowej
            wynik.append(nowy_wiersz)

        # zwrócenie nowej macierzy jako obiektu klasy Macierz
        return Macierz(wynik)

```

```

# funkcja dodająca dwie macierze
def dodawanie(self, inna):
    # sprawdzamy, czy macierze mają taką samą liczbę wierszy
    if len(self.dane) != len(inna.dane):
        raise ValueError("Macierze muszą mieć taką samą liczbę wierszy")

    # sprawdzamy, czy macierze mają taką samą liczbę kolumn
    if len(self.dane[0]) != len(inna.dane[0]):
        raise ValueError("Macierze muszą mieć taką samą liczbę kolumn")

    # pusta lista na wynik
    wynik = []

    # przejście po numerach wierszy
    for i in range(len(self.dane)):
        # nowy wiersz wynikowy
        wiersz = []

        # przejście po numerach kolumn
        for j in range(len(self.dane[0])):
            # dodanie do siebie elementów z obu macierzy o tych samych
indeksach
            wiersz.append(self.dane[i][j] + inna.dane[i][j])

        # dodanie gotowego wiersza do macierzy wynikowej
        wynik.append(wiersz)

    # zwrócenie nowej macierzy jako obiektu klasy Macierz
    return Macierz(wynik)

# funkcja mnożąca dwie macierze
def mnozenie(self, inna):
    # sprawdzamy warunek mnożenia macierzy
    if len(self.dane[0]) != len(inna.dane):
        raise ValueError("Liczba kolumn pierwszej macierzy musi być równa
liczbie wierszy drugiej macierzy")

    # pusta lista na wynik
    wynik = []

    # przejście po wierszach pierwszej macierzy
    for i in range(len(self.dane)):
        # nowy wiersz wynikowy
        wiersz = []

        # przejście po kolumnach drugiej macierzy
        for j in range(len(inna.dane[0])):
            # zmienna przechowująca sumę iloczynów
            suma = 0

            # przejście po elementach wiersza i kolumny
            for k in range(len(self.dane[0])):
                # dodawanie kolejnych iloczynów do sumy

```

```

        suma += self.dane[i][k] * inna.dane[k][j]

        # dodanie obliczonego elementu do wiersza wynikowego
        wiersz.append(suma)

        # dodanie gotowego wiersza do macierzy wynikowej
        wynik.append(wiersz)

        # zwrócenie nowej macierzy jako obiektu klasy Macierz
        return Macierz(wynik)

# utworzenie pierwszej macierzy A
A = Macierz([[1, 2], [3, 4]])

# utworzenie drugiej macierzy B
B = Macierz([[5, 6], [7, 8]])

print("Macierz A:")
A.wypisz()

print("Macierz B:")
B.wypisz()

print("A + B:")
A.dodawanie(B).wypisz()

print("A * 2:")
A.mnozenie_przez_stala(2).wypisz()

print("A * B:")
A.mnozenie(B).wypisz()

```

Lab 4

Zadanie 1

Napisz funkcję zwracającą wyznacznik macierzy kwadratowej dowolnego rozmiaru.

```
import math

def czy_kwadratowa(A):
    if len(A) == 0:
        return False

    liczba_wierszy = len(A)

    for i in range(liczba_wierszy):
        if len(A[i]) != liczba_wierszy:
            return False
    return True

def minor(macierz, usuniety_wiersz, usunieta_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usuniety_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usunieta_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    # sprawdzenie czy jest kwadratowa macierz
    # for wiersz in macierz:
    #     if len(wiersz) != n:
    #         raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie
    # jest kwadratowa")

    if not czy_kwadratowa(macierz):
        raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest
kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    wyznacznik = 0

    for j in range(n):
```

```

        podmacierz = minor(macierz, 0, j)
        wyznacznik += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz)
        # wyznacznik += ((-1) ** j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz) # to żeby nie było float

    return wyznacznik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

print("Wyznacznik macierzy: ", wyznacznik_macierzy(macierz))

```

Wyznacznik macierzy: 66.0

Zadanie 2

Napisz funkcję zwracającą transpozycję macierzy dowolnego rozmiaru.

```
def transpozycja(macierz):  
    liczba_wierszy = len(macierz)  
    liczba_kolumn = len(macierz[0])  
  
    wynik = []  
  
    for j in range(liczba_kolumn):  
        nowy_wiersz = []  
        for i in range(liczba_wierszy):  
            nowy_wiersz.append(macierz[i][j])  
        wynik.append(nowy_wiersz)  
  
    return wynik  
  
macierz = [  
    [2, 4, 6],  
    [0, 2, -1],  
    [-3, 3, 3]  
]  
  
print("Przed transpozycją macierzy:")  
for wiersz in macierz:  
    print(wiersz)  
  
wynik = transpozycja(macierz)  
  
print("Po transpozycji macierzy:")  
for wiersz in wynik:  
    print(wiersz)
```

Przed transpozycją macierzy:

```
[2, 4, 6]  
[0, 2, -1]  
[-3, 3, 3]
```

Po transpozycji macierzy:

```
[2, 0, -3]  
[4, 2, 3]  
[6, -1, 3]
```

Zadanie 3

Napisz funkcję znajdującą macierz odwrotną do macierzy kwadratowej dowolnego rozmiaru za pomocą:

- a) rozwinięcia Laplace'a
- b) metody Gaussa-Jordana

```
import math

def minor(macierz, usun_wiersz, usun_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usun_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usun_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    det = 0
    for j in range(n):
        podmacierz = minor(macierz, 0, j)
        det += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz)

    return det

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []
        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)
```



```

    return wynik

def zeros(n,m):
    macierz = []

    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0)
        macierz.append(wiersz)

    return macierz

def macierz_odwrotna_Laplace(macierz): # punkt a
    n = len(macierz)
    d = wyznacznik_macierzy(macierz)

    if d == 0:
        raise ValueError("Macierz jest osobliwa, nie ma odwrotności")

    if n == 1:
        return [[1 / d]]

    C = zeros(n, n)

    for i in range(n):
        for j in range(n):
            M = minor(macierz, i, j)
            C[i][j] = math.pow(-1, i+j) * wyznacznik_macierzy(M)

    Adj = transpozycja(C)

    wynik = zeros(n, n)
    for i in range(n):
        for j in range(n):
            wynik[i][j] = Adj[i][j] / d

    return wynik

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

```

```

# lewa strona: macierz A
for j in range(n):
    wiersz.append(macierz[i][j])
    # wiersz.append(float(macierz[i][j]))

# prawa strona: macierz jednostkowa I
for j in range(n):
    if i == j:
        wiersz.append(1)
    else:
        wiersz.append(0)

rozszerzona_macierz.append(wiersz)

# algorytm Gaussa-Jordana
for i in range(n):
    # jeśli na przekątnej jest 0, zamień wiersze
    if rozszerzona_macierz[i][i] == 0:
        znaleziono = False
        for k in range(i+1, n):
            if rozszerzona_macierz[k][i] != 0:
                rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i]
                znaleziono = True
                break
        if not znaleziono:
            raise ValueError("Macierz nie ma odwrotności")

    # dzielenie całego wiersza przez element główny
    element_glowny = rozszerzona_macierz[i][i]
    for j in range(2 * n):
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

    # zerowanie pozostałych elementów w tej kolumnie
    for k in range(n):
        if k != i:
            wspolczynnik = rozszerzona_macierz[k][i]
            for j in range(2 * n):
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

    odwrotna = []
    for i in range(n):
        wiersz = []
        for j in range(n, 2 * n):
            wiersz.append(rozszerzona_macierz[i][j])
        odwrotna.append(wiersz)

    return odwrotna

macierz = [
    [2, 4, 6],
    [0, 2, -1],

```

```

    [-3, 3, 3]
]

wynik_Laplace = macierz_odwrotna_Laplace(macierz)
wynik_Gauss_Jordan = macierz_odwrotna_Gaussa_Jordana(macierz)

print("Macierz odwrotna Laplace:")
for wiersz in wynik_Laplace:
    print(wiersz)

print("Macierz odwrotna Gauss Jordan:")
for wiersz in wynik_Gauss_Jordan:
    print(wiersz)

```

Macierz odwrotna Laplace:

```

[0.13636363636363635, 0.09090909090909091, -0.24242424242424243]
[0.045454545454545456, 0.36363636363636365, 0.030303030303030304]
[0.09090909090909091, -0.2727272727272727, 0.06060606060606061]

```

Macierz odwrotna Gauss Jordan:

```

[0.13636363636363635, 0.09090909090909083, -0.24242424242424243]
[0.045454545454545456, 0.36363636363636365, 0.030303030303030304]
[0.09090909090909091, -0.2727272727272727, 0.06060606060606061]

```

Gauss-Jordan do wyznacznika i odwrotnej

```
def macierz_odwrotna_Gaussa_Jordana(macierz):  
    n = len(macierz)  
  
    for wiersz in macierz:  
        if len(wiersz) != n:  
            raise ValueError("Macierz musi być kwadratowa")  
  
    rozszerzona_macierz = []  
  
    for i in range(n):  
        wiersz = []  
  
        for j in range(n):  
            wiersz.append(macierz[i][j])  
  
        for j in range(n):  
            if i == j:  
                wiersz.append(1)  
            else:  
                wiersz.append(0)  
  
        rozszerzona_macierz.append(wiersz)  
  
    wyznacznik = 1  
  
    for i in range(n):  
        if rozszerzona_macierz[i][i] == 0:  
            znaleziono = False  
  
            for k in range(i + 1, n):  
                if rozszerzona_macierz[k][i] != 0:  
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =  
rozszerzona_macierz[k], rozszerzona_macierz[i]  
                    wyznacznik *= -1  
                    znaleziono = True  
                    break  
  
            if not znaleziono:  
                raise ValueError("Macierz nie ma odwrotności")  
  
        element_glowny = rozszerzona_macierz[i][i]  
  
        wyznacznik *= element_glowny  
  
        for j in range(2 * n):  
            rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny  
  
        for k in range(n):  
            if k != i:  
                wspolczynnik = rozszerzona_macierz[k][i]  
  
                for j in range(2 * n):
```

```
        rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -  
wspolczynnik * rozszerzona_macierz[i][j]  
  
    odwrotna = []  
  
    for i in range(n):  
        wiersz = []  
  
        for j in range(n, 2 * n):  
            wiersz.append(rozszerzona_macierz[i][j])  
  
        odwrotna.append(wiersz)  
  
    return odwrotna, wyznacznik
```

Zadanie 4

Napisz funkcję, która wykona mnożenie dwóch macierzy.

```
def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz1 = [
    [1, 2],
    [3, 4]
]

macierz2 = [
    [5, 6],
    [7, 8]
]

wynik = mnozenie_macierzy(macierz1, macierz2)

print("Wynik mnożenia:")
for wiersz in wynik:
    print(wiersz)

macierzAB = mnozenie_macierzy(macierz1, macierz2)
macierzBA = mnozenie_macierzy(macierz2, macierz1)
```

Wynik mnożenia:
 [19, 22]
 [43, 50]

Zadanie 5

Korzystając z rozwiązań poprzednich zadań wykonaj następujące mnożenia macierzowe: $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$ i porównaj ich wyniki.

```

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        # lewa strona: macierz A
        for j in range(n):
            wiersz.append(macierz[i][j])
            # wiersz.append(float(macierz[i][j]))

        # prawa strona: macierz jednostkowa I
        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):
        # jeśli na przekątnej jest 0, zamień wiersze
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
                    rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):
            rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

        # zerowanie pozostałych elementów w tej kolumnie
        for k in range(n):
            if k != i:
                wspolczynnik = rozszerzona_macierz[k][i]
                for j in range(2 * n):
                    rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -

```

```

wspolczynnik * rozszerzona_macierz[i][j]

odwrotna = []
for i in range(n):
    wiersz = []
    for j in range(n, 2 * n):
        wiersz.append(rozszerzona_macierz[i][j])
    odwrotna.append(wiersz)

return odwrotna

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz1 = [
    [1, 2],
    [3, 4]
]

macierz2 = [
    [5, 6],
    [7, 8]
]

wynik = mnozenie_macierzy(macierz1, macierz2)

print("Wynik mnożenia:")
for wiersz in wynik:
    print(wiersz)

macierzAB = mnozenie_macierzy(macierz1, macierz2)
macierzBA = mnozenie_macierzy(macierz2, macierz1)

macierz = [

```



```

    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

macierz_odwrotna = macierz_odwrotna_Gausa_Jordana(macierz)

wynik1 = mnozenie_macierzy(macierz, macierz_odwrotna)
wynik2 = mnozenie_macierzy(macierz_odwrotna, macierz)

print("Wynik mnożenia A * A^-1:")
for wiersz in wynik1:
    print(wiersz)

print("Wynik mnożenia A^-1 * A:")
for wiersz in wynik2:
    print(wiersz)

```

```

Wynik mnożenia A * A^-1:
[1.0, 0.0, 0.0]
[0.0, 1.0, 0.0]
[0.0, 2.220446049250313e-16, 1.0]

Wynik mnożenia A^-1 * A:
[1.0, -2.220446049250313e-16, 0.0]
[0.0, 1.0, -2.7755575615628914e-17]
[0.0, 5.551115123125783e-17, 1.0]

```

Lab 5

Zadanie 1

Korzystając z funkcji napisanych na poprzednich zajęciach, rozwiąż następujące układy równań liniowych:

a)

$$\begin{aligned}
 x + 2y + z &= -9, \\
 3x - 7y + 2z &= 61, \\
 2x + 4y + 5z &= -9.
 \end{aligned}$$

b)

$$Ax = b,$$

gdzie:

$$A \in \mathbb{R}^{n \times n}, \quad b \in \mathbb{R}^{n \times 1}, \quad n = \{8, 10\}$$

oraz:

$$A = \begin{bmatrix} 11 & -5 & 0 & & 0 \\ -5 & 11 & -5 & \ddots & \vdots \\ 0 & -5 & 11 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -5 \\ 0 & & 0 & -5 & 11 \end{bmatrix}, \quad b = \begin{bmatrix} 11 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

c)

Układ równań ze współczynnikami tworzącymi macierz gęstą rozmiaru:

$$10 \times 10$$

Zmierz czas potrzebny na znalezienie każdego z rozwiązań.

```

import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        # lewa strona: macierz A
        for j in range(n):
            wiersz.append(macierz[i][j])
            # wiersz.append(float(macierz[i][j]))

        # prawa strona: macierz jednostkowa I
        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):
        # jeśli na przekątnej jest 0, zamień wiersze
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
                    rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):

```

```

        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

# zerowanie pozostałych elementów w tej kolumnie
for k in range(n):
    if k != i:
        wspolczynnik = rozszerzona_macierz[k][i]
        for j in range(2 * n):
            rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

odwrotna = []
for i in range(n):
    wiersz = []
    for j in range(n, 2 * n):
        wiersz.append(rozszerzona_macierz[i][j])
    odwrotna.append(wiersz)

return odwrotna

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

def rozwiaz_uklad_rownan(macierz_A, wektor_b):
    b_macierz = [[b] for b in wektor_b]

    A_odwrotna = macierz_odwrotna_Gaussa_Jordana(macierz_A)

    wynik = mnozenie_macierzy(A_odwrotna, b_macierz)

    return wynik

```

Podpunkt a)

```
A = [  
    [1, 2, 1],  
    [3, -7, 2],  
    [2, 4, 5]  
]  
  
b = [-9, 61, -9]  
  
wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, A, b)  
  
print("-----ZADANIE 1-----")  
  
print("Rozwiązanie układu równań a:")  
print("x =", wynik[0][0])  
print("y =", wynik[1][0])  
print("z =", wynik[2][0])  
print("Czas: ", czas)
```

Podpunkt b)

```
def zeros(n,m):
    macierz = []

    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0)
        macierz.append(wiersz)

    return macierz

def tworzenie_macierzy(n):
    A = zeros(n, n)
    b = [11] + [0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11
            elif abs(i - j) == 1:
                A[i][j] = -5

    return A, b

n_8_A, n_8_b = tworzenie_macierzy(8)
n_10_A, n_10_b = tworzenie_macierzy(10)

wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, n_8_A, n_8_b)
print("Rozwiązanie układu równań b) n=8:")
for i in range(len(wynik)):
    print(wynik[i][0])
print("Czas: ", czas)

wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, n_10_A, n_10_b)
print("Rozwiązanie układu równań b) n=10:")
for i in range(len(wynik)):
    print(wynik[i][0])
print("Czas: ", czas)
```

Podpunkt c)

```
def tworzenie_macierzy_gestej(n, m):
    A = zeros(n, m)

    for i in range(n):
        for j in range(m):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

    x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

    x_macierz = [[x] for x in x_prawdziwe]
    b_macierz = mnozenie_macierzy(A, x_macierz)

    b = []
    for i in range(len(b_macierz)):
        b.append(b_macierz[i][0])

    return A, b, x_prawdziwe

macierz, wektor, x = tworzenie_macierzy_gestej(10, 10)

wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, macierz, wektor)

print("Rozwiązanie układu równań c:")
for i in range(len(wynik)):
    print(wynik[i])
print("Czas: ", czas)
```

Inne sposoby rozwiązania bez liczenia macierzy odwrotnej

$$x = A^{-1} b$$

1. Eliminacja Gaussa — metoda bezpośrednia.
2. Gauss-Jordan na macierzy rozszerzonej $[A|b]$ — metoda bezpośrednia.
3. Metoda Jacobiego — metoda iteracyjna z wykładu.
4. Metoda Gaussa-Seidla — metoda iteracyjna z wykładu.

```
import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

def kopiuj_macierz(A):
    kopia = []

    for wiersz in A:
        kopia.append(wiersz.copy())

    return kopia

def zeros(n, m):
    macierz = []

    for i in range(n):
        wiersz = []

        for j in range(m):
            wiersz.append(0.0)

        macierz.append(wiersz)

    return macierz

def sprawdz_uklad(A, b):
    n = len(A)

    if len(A) == 0:
        raise ValueError("Macierz A nie może być pusta")

    if len(b) != n:
        raise ValueError("Wektor b musi mieć tyle elementów, ile macierz A ma wierszy")

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz A musi być kwadratowa")

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
```



```

ilosc_kolumn_macierz2 = len(macierz2[0])

if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
    raise ValueError("Nie da się pomnożyć tych macierzy")

wynik = []

for i in range(ilosc_wierszy_macierz1):
    wiersz = []

    for j in range(ilosc_kolumn_macierz2):
        suma = 0.0

        for k in range(ilosc_kolumn_macierz1):
            suma += macierz1[i][k] * macierz2[k][j]

        wiersz.append(suma)

    wynik.append(wiersz)

return wynik

```

```

# -----
# METODA 1: ELIMINACJA GAUSSA
# -----

```

```

def rozwarz_eliminacja_Gaussa(A, b):
    sprawdz_uklad(A, b)

    A = kopiuuj_macierz(A)
    b = b.copy()

    n = len(A)

    # eliminacja w przód
    for i in range(n):

        # szukanie najlepszego pivota w kolumnie i
        max_wiersz = i

        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_wiersz][i]):
                max_wiersz = k

        # jeśli największy element w kolumnie jest zerem,
        # to układ nie ma jednoznacznego rozwiązania
        if A[max_wiersz][i] == 0:
            raise ValueError("Układ nie ma jednoznacznego rozwiązania")

        # zamiana wierszy, jeśli trzeba
        if max_wiersz != i:
            A[i], A[max_wiersz] = A[max_wiersz], A[i]

```

```

        b[i], b[max_wiersz] = b[max_wiersz], b[i]

# zerowanie elementów pod przekątną
for j in range(i + 1, n):
    wspolczynnik = A[j][i] / A[i][i]

    for k in range(i, n):
        A[j][k] = A[j][k] - wspolczynnik * A[i][k]

    b[j] = b[j] - wspolczynnik * b[i]

# podstawianie wsteczne
x = [0.0] * n

for i in range(n - 1, -1, -1):
    suma = 0.0

    for j in range(i + 1, n):
        suma += A[i][j] * x[j]

    x[i] = (b[i] - suma) / A[i][i]

return x

# -----
# METODA 2: GAUSS-JORDAN NA MACIERZY [A | b]
# -----

def rozwarz_Gauss_Jordan(A, b):
    sprawdz_uklad(A, b)

    n = len(A)

    # tworzymy macierz rozszerzoną [A | b]
    rozszerzona = []

    for i in range(n):
        wiersz = []

        for j in range(n):
            wiersz.append(float(A[i][j]))

        wiersz.append(float(b[i]))

        rozszerzona.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):

        # szukanie najlepszego pivota w kolumnie i
        max_wiersz = i

```

```

        for k in range(i + 1, n):
            if abs(rozszerzona[k][i]) > abs(rozszerzona[max_wiersz][i]):
                max_wiersz = k

        if rozszerzona[max_wiersz][i] == 0:
            raise ValueError("Układ nie ma jednoznacznego rozwiązania")

        # zamiana wierszy, jeśli trzeba
        if max_wiersz != i:
            rozszerzona[i], rozszerzona[max_wiersz] = rozszerzona[max_wiersz],
rozszerzona[i]

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona[i][i]

        for j in range(n + 1):
            rozszerzona[i][j] = rozszerzona[i][j] / element_glowny

        # zerowanie pozostałych elementów w tej kolumnie
        for k in range(n):
            if k != i:
                wspolczynnik = rozszerzona[k][i]

                for j in range(n + 1):
                    rozszerzona[k][j] = rozszerzona[k][j] - wspolczynnik *
rozszerzona[i][j]

        # ostatnia kolumna to rozwiązanie układu
        wynik = []

        for i in range(n):
            wynik.append(rozszerzona[i][n])

        return wynik

# -----
# FUNKCJE POMOCNICZE DO METOD ITERACYJNYCH
# -----

def norma_maksimum_wektora(wektor):
    maksimum = 0.0

    for element in wektor:
        if abs(element) > maksimum:
            maksimum = abs(element)

    return maksimum

def roznica_wektorow(x, y):
    wynik = []

```

```

for i in range(len(x)):
    wynik.append(x[i] - y[i])

return wynik

def sprawdz_diagonale(A):
    n = len(A)

    for i in range(n):
        if A[i][i] == 0:
            raise ValueError("Na przekątnej znajduje się zero")

def norma_wierszowa_W(A):
    sprawdz_diagonale(A)

    n = len(A)
    maksimum = 0.0

    for i in range(n):
        suma = 0.0

        for j in range(n):
            if i != j:
                suma += abs(-A[i][j] / A[i][i])

        if suma > maksimum:
            maksimum = suma

    return maksimum

def wypisz_warunek_iteracyjny(A):
    try:
        norma = norma_wierszowa_W(A)

        print("Norma wierszowa macierzy W =", norma)

        if norma < 1:
            print("Warunek  $\|W\| < 1$  jest spełniony, metoda iteracyjna powinna być zbieżna")
        else:
            print("Warunek  $\|W\| < 1$  nie jest spełniony, metoda iteracyjna może nie być zbieżna")

    except ValueError as blad:
        print("Nie można sprawdzić warunku iteracyjnego:", blad)

    print()

```

```

# METODA 3: JACOBI
# -----

def rozwarz_Jacobi(A, b):
    sprawdz_uklad(A, b)
    sprawdz_diagonale(A)

    epsilon = 1e-3
    max_iter = 1000

    n = len(A)

    x_stare = []

    for i in range(n):
        x_stare.append(0.0)

    for iteracja in range(max_iter):
        x_nowe = []

        for i in range(n):
            suma = 0.0

            for j in range(n):
                if j != i:
                    suma += A[i][j] * x_stare[j]

            x_i = (b[i] - suma) / A[i][i]
            x_nowe.append(x_i)

        roznica = roznica_wektorow(x_nowe, x_stare)

        norma_roznicy = norma_maksimum_wektora(roznica)
        norma_x = norma_maksimum_wektora(x_nowe)

        if norma_x != 0:
            blad = norma_roznicy / norma_x
        else:
            blad = norma_roznicy

        if blad <= epsilon:
            return x_nowe

        x_stare = x_nowe.copy()

    raise ValueError("Metoda Jacobiego nie osiągnęła wymaganej dokładności")

# -----
# METODA 4: GAUSS-SEIDEL
# -----

def rozwarz_Gauss_Seidel(A, b):

```

```

sprawdz_uklad(A, b)
sprawdz_diagonale(A)

epsilon = 1e-3
max_iter = 1000

n = len(A)

x = []

for i in range(n):
    x.append(0.0)

for iteracja in range(max_iter):
    x_stare = x.copy()

    for i in range(n):
        suma = 0.0

        for j in range(n):
            if j != i:
                suma += A[i][j] * x[j]

        x[i] = (b[i] - suma) / A[i][i]

    roznica = roznica_wektorow(x, x_stare)

    norma_roznicy = norma_maksimum_wektora(roznica)
    norma_x = norma_maksimum_wektora(x)

    if norma_x != 0:
        blad = norma_roznicy / norma_x
    else:
        blad = norma_roznicy

    if blad <= epsilon:
        return x

    raise ValueError("Metoda Gaussa-Seidla nie osiągnęła wymaganej dokładności")

# -----
# TWORZENIE DANYCH DO ZADAŃ
# -----

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0

```

```

        elif abs(i - j) == 1:
            A[i][j] = -5.0
        else:
            A[i][j] = 0.0

    return A, b

def tworzenie_macierzy_gestej(n):
    A = zeros(n, n)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

    x_prawdziwe = []

    for i in range(n):
        x_prawdziwe.append(float(i + 1))

    x_macierz = []

    for x in x_prawdziwe:
        x_macierz.append([x])

    b_macierz = mnozenie_macierzy(A, x_macierz)

    b = []

    for i in range(len(b_macierz)):
        b.append(b_macierz[i][0])

    return A, b, x_prawdziwe

# -----
# WYPISYWANIE WYNIKÓW
# -----

def wypisz_wynik(nazwa, wynik, czas):
    print(nazwa)

    for i in range(len(wynik)):
        print("x_", i + 1, "=", wynik[i])

    print("Czas:", czas)
    print()

def uruchom_metode(nazwa, funkcja, A, b):

```

```

try:
    wynik, czas = zmierz_czas(funkcja, A, b)
    wypisz_wynik(nazwa, wynik, czas)

except ValueError as blad:
    print(nazwa)
    print("Nie udało się zastosować metody:", blad)
    print()

# -----
# ZADANIE 1
# -----

print("-----ZADANIE 1-----")
print()

# -----
# PUNKT a)
# -----

print("PUNKT a)")
print()

A_a = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]

b_a = [-9.0, 61.0, -9.0]

wypisz_warunek_iteracyjny(A_a)

uruchom_metode("Eliminacja Gaussa:", rozwarz_eliminacja_Gaussa, A_a, b_a)
uruchom_metode("Gauss-Jordan [A | b]:", rozwarz_Gauss_Jordan, A_a, b_a)
uruchom_metode("Metoda Jacobiego:", rozwarz_Jacobi, A_a, b_a)
uruchom_metode("Metoda Gaussa-Seidla:", rozwarz_Gauss_Seidel, A_a, b_a)

# -----
# PUNKT b) n = 8
# -----

print("PUNKT b) n = 8")
print()

A_b8, b_b8 = tworzenie_macierzy_b(8)

wypisz_warunek_iteracyjny(A_b8)

uruchom_metode("Eliminacja Gaussa:", rozwarz_eliminacja_Gaussa, A_b8, b_b8)

```



```

uruchom_metode("Gauss-Jordan [A | b]:", rozwiaz_Gauss_Jordan, A_b8, b_b8)
uruchom_metode("Metoda Jacobiego:", rozwiaz_Jacobi, A_b8, b_b8)
uruchom_metode("Metoda Gaussa-Seidla:", rozwiaz_Gauss_Seidel, A_b8, b_b8)

# -----
# PUNKT b) n = 10
# -----

print("PUNKT b) n = 10")
print()

A_b10, b_b10 = tworzenie_macierzy_b(10)

wypisz_warunek_iteracyjny(A_b10)

uruchom_metode("Eliminacja Gaussa:", rozwiaz_eliminacja_Gaussa, A_b10, b_b10)
uruchom_metode("Gauss-Jordan [A | b]:", rozwiaz_Gauss_Jordan, A_b10, b_b10)
uruchom_metode("Metoda Jacobiego:", rozwiaz_Jacobi, A_b10, b_b10)
uruchom_metode("Metoda Gaussa-Seidla:", rozwiaz_Gauss_Seidel, A_b10, b_b10)

# -----
# PUNKT c)
# -----

print("PUNKT c)")
print()

A_c, b_c, x_prawdziwe = tworzenie_macierzy_gestej(10)

wypisz_warunek_iteracyjny(A_c)

uruchom_metode("Eliminacja Gaussa:", rozwiaz_eliminacja_Gaussa, A_c, b_c)
uruchom_metode("Gauss-Jordan [A | b]:", rozwiaz_Gauss_Jordan, A_c, b_c)
uruchom_metode("Metoda Jacobiego:", rozwiaz_Jacobi, A_c, b_c)
uruchom_metode("Metoda Gaussa-Seidla:", rozwiaz_Gauss_Seidel, A_c, b_c)

print("Wartości prawdziwe użyte do utworzenia wektora b w punkcie c):")
for i in range(len(x_prawdziwe)):
    print("x_", i + 1, "=", x_prawdziwe[i])

```

Zadanie 2

Napisz funkcję znajdującą rozkład macierzy na iloczyn macierzy trójkątnych $A = LU$. Powyższą funkcję wykorzystaj w celu znalezienia rozwiązania układu równań liniowych. Przetestuj działanie dla przykładów z zadania 1. Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

metoda Doolittle'a bez pivotowania

```

import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def rozklad_LU_Doolittle(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    L = zeros(n, n)
    U = zeros(n, n)

    # na przekątnej L są jedynki
    for i in range(n):
        L[i][i] = 1.0

    for i in range(n):
        # liczenie elementów U
        for j in range(i, n):
            suma = 0.0
            for k in range(i):
                suma += L[i][k] * U[k][j]
            U[i][j] = A[i][j] - suma

        # liczenie elementów L
        for j in range(i + 1, n):
            suma = 0.0
            for k in range(i):
                suma += L[j][k] * U[k][i]

            if U[i][i] == 0:
                raise ValueError("Nie można wykonać rozkładu LU metodą Doolittle'a")

            L[j][i] = (A[j][i] - suma) / U[i][i]

    return L, U

```

```
def podstawianie_w_przod(L, b):  
    n = len(L)  
    y = [0.0] * n  
  
    for i in range(n):  
        suma = 0.0  
        for j in range(i):  
            suma += L[i][j] * y[j]  
        y[i] = b[i] - suma  
  
    return y
```

```
def podstawianie_w tyl(U, y):  
    n = len(U)  
    x = [0.0] * n  
  
    for i in range(n - 1, -1, -1):  
        suma = 0.0  
        for j in range(i + 1, n):  
            suma += U[i][j] * x[j]  
  
        if U[i][i] == 0:  
            raise ValueError("Dzielenie przez zero w podstawianiu w tyl")  
  
        x[i] = (y[i] - suma) / U[i][i]  
  
    return x
```

```
def rozwarz_uklad_Doolittle(A, b):  
    L, U = rozklad_LU_Doolittle(A)  
    y = podstawianie_w_przod(L, b)  
    x = podstawianie_w tyl(U, y)  
    return x
```

```

#punkt a

A = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]

b = [-9.0, 61.0, -9.0]

wynik, czas = zmierz_czas(rozwiaz_uklad_Doolittle, A, b)

print("Rozwiązanie metodą Doolittle'a dla a:")
for i in range(len(wynik)):
    print("x" + str(i + 1) + " =", wynik[i])
print("Czas: ", czas)

#punkt b

def wypisz_wektor(wektor):
    for i in range(len(wektor)):
        print("x" + str(i + 1) + " =", wektor[i])

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0
            elif abs(i - j) == 1:
                A[i][j] = -5.0
            else:
                A[i][j] = 0.0

    return A, b

#punkt b, n = 8

A8, b8 = tworzenie_macierzy_b(8)

wynik8, czas8 = zmierz_czas(rozwiaz_uklad_Doolittle, A8, b8)

print("\nRozwiązanie metodą Doolittle'a dla b), n=8:")
wypisz_wektor(wynik8)
print("Czas:", czas8, "s")

#punkt b, n = 10

A10, b10 = tworzenie_macierzy_b(10)

```

```

wynik10, czas10 = zmierz_czas(rozwiaz_uklad_Doolittle, A10, b10)

print("\nRozwiązanie metodą Doolittle'a dla b), n=10:")
wypisz_wektor(wynik10)
print("Czas:", czas10, "s")

#punkt c

def wektor_na_macierz_kolumnowa(wektor):
    wynik = []
    for x in wektor:
        wynik.append([float(x)])
    return wynik

def macierz_kolumnowa_na_wektor(macierz):
    wynik = []
    for i in range(len(macierz)):
        wynik.append(macierz[i][0])
    return wynik

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0.0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

def tworzenie_macierzy_gestej(n, m):
    A = zeros(n, m)

    for i in range(n):
        for j in range(m):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

```

```
x_macierz = wektor_na_macierz_kolumnowa(x_prawdziwe)
b_macierz = mnozenie_macierzy(A, x_macierz)
b = macierz_kolumnowa_na_wektor(b_macierz)

return A, b, x_prawdziwe

Ag, bg, x_prawdziwe = tworzenie_macierzy_gestej(10, 10)

wynikg, czasg = zmierz_czas(rozwiaz_uklad_Doolittle, Ag, bg)

print("\nRozwiązanie metodą Doolittle'a dla c:")
wypisz_wektor(wynikg)
print("Czas:", czasg, "s")
print("Oczekiwane rozwiązanie:", x_prawdziwe)
```

metoda Doolittle'a z pivotowaniem częściowym

```

def kopiuj_macierz(macierz):
    wynik = []
    for wiersz in macierz:
        nowy_wiersz = []
        for element in wiersz:
            nowy_wiersz.append(float(element))
        wynik.append(nowy_wiersz)
    return wynik

def rozklad_LU_Doolittle(A):
    n = len(A)
    U = kopiuj_macierz(A)
    L = zeros(n, n)
    P = list(range(n))
    eps = 1e-12

    for i in range(n):
        # wybór pivota
        max_wiersz = i
        max_wartosc = abs(U[i][i])

        for k in range(i + 1, n):
            if abs(U[k][i]) > max_wartosc:
                max_wartosc = abs(U[k][i])
                max_wiersz = k

        if max_wartosc < eps:
            raise ValueError("Nie można wykonać rozkładu LU - macierz osobliwa")

        # zamiana wierszy w U
        if max_wiersz != i:
            U[i], U[max_wiersz] = U[max_wiersz], U[i]
            P[i], P[max_wiersz] = P[max_wiersz], P[i]

        # w L zamieniamy tylko elementy już policzone
        for j in range(i):
            L[i][j], L[max_wiersz][j] = L[max_wiersz][j], L[i][j]

        L[i][i] = 1.0

        # zerowanie pod przekątną
        for j in range(i + 1, n):
            L[j][i] = U[j][i] / U[i][i]

            for k in range(i, n):
                U[j][k] = U[j][k] - L[j][i] * U[i][k]

    return L, U, P

def permutuj_wektor(b, P):
    return [float(b[P[i]]) for i in range(len(P))]

```

```

def podstawianie_w_przod(L, b):
    n = len(L)
    y = [0.0] * n

    for i in range(n):
        suma = 0.0
        for j in range(i):
            suma += L[i][j] * y[j]
        y[i] = b[i] - suma

    return y

def podstawianie_w tyl(U, y):
    n = len(U)
    x = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0
        for j in range(i + 1, n):
            suma += U[i][j] * x[j]

        if U[i][i] == 0:
            raise ValueError("Dzielenie przez zero w podstawianiu w tyl")

        x[i] = (y[i] - suma) / U[i][i]

    return x

def rozwarz_uklad_Doolittle(A, b):
    L, U, P = rozklad_LU_Doolittle(A)
    pb = permutuj_wektor(b, P)
    y = podstawianie_w_przod(L, pb)
    x = podstawianie_w tyl(U, y)
    return x

```

LU z eliminacji Gaussa


```
def kopiuj_macierz(macierz):
    wynik = []

    for wiersz in macierz:
        nowy_wiersz = []

        for element in wiersz:
            nowy_wiersz.append(float(element))

        wynik.append(nowy_wiersz)

    return wynik


def rozklad_LU_Gauss(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    U = kopiuj_macierz(A)
    L = zeros(n, n)

    # na przekątnej L są jedynki
    for i in range(n):
        L[i][i] = 1.0

    for i in range(n):
        if U[i][i] == 0:
            raise ValueError("Nie można wykonać rozkładu LU bez zamiany wierszy")

        for j in range(i + 1, n):
            wspolczynnik = U[j][i] / U[i][i]

            L[j][i] = wspolczynnik

            for k in range(i, n):
                U[j][k] = U[j][k] - wspolczynnik * U[i][k]

    return L, U


def rozwarz_uklad_LU_Gauss(A, b):
    L, U = rozklad_LU_Gauss(A)

    y = podstawianie_w_przod(L, b)

    x = podstawianie_w tyl(U, y)

    return x
```

```
wynik, czas = zmierz_czas(rozwiaz_uklad_LU_Gauss, A, b)

print("Rozwiązanie metodą LU z eliminacji Gaussa dla a:")
for i in range(len(wynik)):
    print("x" + str(i + 1) + " =", wynik[i])
print("Czas:", czas)
```

Metoda Crouta

W metodzie Crouta też mamy: $A = LU$ ale tym razem macierz U ma jedynki na przekątnej.

```

def rozklad_LU_Crout(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    L = zeros(n, n)
    U = zeros(n, n)

    # na przekątnej U są jedynki
    for i in range(n):
        U[i][i] = 1.0

    for j in range(n):
        # liczenie elementów L
        for i in range(j, n):
            suma = 0.0

            for k in range(j):
                suma += L[i][k] * U[k][j]

            L[i][j] = A[i][j] - suma

        if L[j][j] == 0:
            raise ValueError("Nie można wykonać rozkładu LU metodą Crouta")

    # liczenie elementów U
    for i in range(j + 1, n):
        suma = 0.0

        for k in range(j):
            suma += L[j][k] * U[k][i]

        U[j][i] = (A[j][i] - suma) / L[j][j]

    return L, U

def rozwarz_uklad_Crout(A, b):
    L, U = rozklad_LU_Crout(A)

    y = podstawianie_w_przod(L, b)

    x = podstawianie_w tyl(U, y)

    return x

```

```
wynik, czas = zmierz_czas(rozwiaz_uklad_Crout, A, b)

print("Rozwiązanie metodą Crouta dla a:")
for i in range(len(wynik)):
    print("x" + str(i + 1) + " =", wynik[i])
print("Czas:", czas)
```

Metoda Cholesky'ego/Banachiewicza

To metoda z wykładu, ale jako **przypadek specjalny**. Działa tylko wtedy, gdy macierz jest symetryczna i dodatnio określona. Wtedy:

$$A = LL^T$$

Wykład podaje, że dla takiej macierzy istnieje rozkład $A = LL^T$, a metoda jest bardziej wydajna od LU i stabilna numerycznie.

```
import math

def czy_symetryczna(A):
    n = len(A)

    for i in range(n):
        for j in range(n):
            if abs(A[i][j] - A[j][i]) > 1e-12:
                return False

    return True

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []

        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def rozklad_Choleskiego(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    if not czy_symetryczna(A):
        raise ValueError("Macierz nie jest symetryczna")

    L = zeros(n, n)

    for i in range(n):
        for j in range(i + 1):
            suma = 0.0

            for k in range(j):
                suma += L[i][k] * L[j][k]

            if i == j:
                wartosc = A[i][i] - suma
```

```

        if wartosc <= 0:
            raise ValueError("Macierz nie jest dodatnio określona")

        L[i][j] = math.sqrt(wartosc)

    else:
        if L[j][j] == 0:
            raise ValueError("Dzielenie przez zero w rozkładzie
Choleskiego")

        L[i][j] = (A[i][j] - suma) / L[j][j]

    return L

def rozwarz_uklad_Cholesky(A, b):
    L = rozklad_Choleskiego(A)

    LT = transpozycja(L)

    y = podstawianie_w_przod(L, b)

    x = podstawianie_w tyl(LT, y)

    return x

```

```

wynik, czas = zmierz_czas(rozwarz_uklad_Cholesky, A, b)

print("Rozwiązanie metodą Cholesky'ego:")
for i in range(len(wynik)):
    print("x" + str(i + 1) + " =", wynik[i])
print("Czas:", czas)

```

Zadanie 3

Napisz funkcję znajdującą rozkład Choleskiego dla macierzy kwadratowej, symetrycznej i dodatnio określonej. Powyższą funkcję wykorzystaj w celu znalezienia rozwiązania układu równań liniowych. Przetestuj działanie dla przykładu b) z zadania 1. Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

```
import time

def czy_symetryczna(A, eps=1e-12):
    n = len(A)

    for i in range(n):
        for j in range(n):
            if abs(A[i][j] - A[j][i]) > eps:
                return False

    return True

def zeros(n, m):
    macierz = []

    for i in range(n):
        wiersz = []

        for j in range(m):
            wiersz.append(0.0)

        macierz.append(wiersz)

    return macierz

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []

        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def podstawianie_w_przod(L, b):
    n = len(L)
    y = [0.0] * n

    for i in range(n):
        suma = 0.0
```

```

    for j in range(i):
        suma += L[i][j] * y[j]

    if L[i][i] == 0:
        raise ValueError("Dzielenie przez zero w podstawianiu w przód")

    y[i] = (b[i] - suma) / L[i][i]

    return y

def podstawianie_w tyl(U, y):
    n = len(U)
    x = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0

        for j in range(i + 1, n):
            suma += U[i][j] * x[j]

        if U[i][i] == 0:
            raise ValueError("Dzielenie przez zero w podstawianiu w tył")

        x[i] = (y[i] - suma) / U[i][i]

    return x

def rozklad_Choleskiego(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    if not czy_symetryczna(A):
        raise ValueError("Macierz musi być symetryczna")

    L = zeros(n, n)

    for i in range(n):
        for j in range(i + 1):
            suma = 0.0

            for k in range(j):
                suma += L[i][k] * L[j][k]

            if i == j:
                wartosc = A[i][i] - suma

                if wartosc <= 0:
                    raise ValueError("Macierz nie jest dodatnio określona")

```



```

        L[i][j] = wartosc ** 0.5

    else:
        if L[j][j] == 0:
            raise ValueError("Dzielenie przez zero w rozkładzie
Choleskiego")

        L[i][j] = (A[i][j] - suma) / L[j][j]

    return L

def rozwiaz_uklad_Choleski(A, b):
    L = rozklad_Choleskiego(A)
    Lt = transpozycja(L)

    # Najpierw rozwiązujemy  $Ly = b$ 
    y = podstawianie_w_przod(L, b)

    # Potem rozwiązujemy  $L^T x = y$ 
    x = podstawianie_w tyl(Lt, y)

    return x

def mnozenie_macierzy_przez_wektor(A, x):
    wynik = []

    for i in range(len(A)):
        suma = 0.0

        for j in range(len(A[i])):
            suma += A[i][j] * x[j]

        wynik.append(suma)

    return wynik

def blad_rozwiazania(A, x, b):
    Ax = mnozenie_macierzy_przez_wektor(A, x)

    maksimum = 0.0

    for i in range(len(b)):
        roznica = abs(Ax[i] - b[i])

        if roznica > maksimum:
            maksimum = roznica

    return maksimum

```

```

def wypisz_wektor(wektor):
    for i in range(len(wektor)):
        print("x" + str(i + 1) + " =", wektor[i])

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0
            elif abs(i - j) == 1:
                A[i][j] = -5.0
            else:
                A[i][j] = 0.0

    return A, b

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()

    return wynik, koniec - start

print("-----ZADANIE 3-----")

# przykład b) z zadania 1, n = 8
A8, b8 = tworzenie_macierzy_b(8)

wynik8, czas8 = zmierz_czas(rozwiaz_uklad_Choleski, A8, b8)

print("Rozwiązanie metodą Cholesky'ego dla n = 8:")
wypisz_wektor(wynik8)
print("Czas:", czas8)
print("Błąd ||Ax - b|| =", blad_rozwiazania(A8, wynik8, b8))

# przykład b) z zadania 1, n = 10
A10, b10 = tworzenie_macierzy_b(10)

wynik10, czas10 = zmierz_czas(rozwiaz_uklad_Choleski, A10, b10)

print("\nRozwiązanie metodą Cholesky'ego dla n = 10:")
wypisz_wektor(wynik10)
print("Czas:", czas10)
print("Błąd ||Ax - b|| =", blad_rozwiazania(A10, wynik10, b10))

```

```
print("\nPorównanie:")
print("Czas dla n = 8:", czas8)
print("Czas dla n = 10:", czas10)
print("Błąd dla n = 8:", blad_rozwiazania(A8, wynik8, b8))
print("Błąd dla n = 10:", blad_rozwiazania(A10, wynik10, b10))
```

Zadanie 4

Napisz funkcję rozwiązującą układy równań liniowych za pomocą eliminacji Gaussa. Przetestuj działanie dla przykładów z zadania 1. Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

eliminacja Gaussa z pivotingiem częściowym

```
import time

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def kopiuj_macierz(macierz):
    wynik = []
    for wiersz in macierz:
        nowy_wiersz = []
        for element in wiersz:
            nowy_wiersz.append(float(element))
        wynik.append(nowy_wiersz)
    return wynik

def wypisz_wektor(wektor):
    for i in range(len(wektor)):
        print("x" + str(i + 1) + " =", wektor[i])

def rozwarz_uklad_Gaussa(A, b):
    n = len(A)
    M = kopiuj_macierz(A)
    bb = []
    for x in b:
        bb.append(float(x))
    eps = 1e-12

    for i in range(n):
        # wybór najlepszego pivota w kolumnie i
        max_wiersz = i
        max_wartosc = abs(M[i][i])

        for k in range(i + 1, n):
            if abs(M[k][i]) > max_wartosc:
                max_wartosc = abs(M[k][i])
                max_wiersz = k

        if max_wartosc < eps:
            raise ValueError("Układ nie ma jednoznacznego rozwiązania")

        # zamiana wierszy
        if max_wiersz != i:
            M[i], M[max_wiersz] = M[max_wiersz], M[i]
            bb[i], bb[max_wiersz] = bb[max_wiersz], bb[i]
```

```

# eliminacja w przód
for k in range(i + 1, n):
    wspolczynnik = M[k][i] / M[i][i]

    for j in range(i, n):
        M[k][j] = M[k][j] - wspolczynnik * M[i][j]

    bb[k] = bb[k] - wspolczynnik * bb[i]

# podstawianie w tył
x = [0.0] * n
for i in range(n - 1, -1, -1):
    suma = 0.0
    for j in range(i + 1, n):
        suma += M[i][j] * x[j]

    if abs(M[i][i]) < eps:
        raise ValueError("Dzielenie przez zero w podstawianiu w tył")

    x[i] = (bb[i] - suma) / M[i][i]

return x

```

#a)

```

A1 = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]

```

```

b1 = [-9.0, 61.0, -9.0]

```

#b)

```

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0
            elif abs(i - j) == 1:
                A[i][j] = -5.0
            else:
                A[i][j] = 0.0

    return A, b

```

#c)

```

def mnozenie_macierzy(macierz1, macierz2):
    liczba_wierszy_1 = len(macierz1)
    liczba_wierszy_2 = len(macierz2)
    liczba_kolumn_1 = len(macierz1[0])
    liczba_kolumn_2 = len(macierz2[0])

    if liczba_kolumn_1 != liczba_wierszy_2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(liczba_wierszy_1):
        wiersz = []
        for j in range(liczba_kolumn_2):
            suma = 0.0
            for k in range(liczba_kolumn_1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

def wektor_na_macierz_kolumnowa(wektor):
    wynik = []
    for x in wektor:
        wynik.append([float(x)])
    return wynik

def macierz_kolumnowa_na_wektor(macierz):
    wynik = []
    for i in range(len(macierz)):
        wynik.append(macierz[i][0])
    return wynik

def tworzenie_macierzy_gestej_10():
    A = zeros(10, 10)

    for i in range(10):
        for j in range(10):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

    x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

    x_kolumna = wektor_na_macierz_kolumnowa(x_prawdziwe)
    b_kolumna = mnozenie_macierzy(A, x_kolumna)
    b = macierz_kolumnowa_na_wektor(b_kolumna)

    return A, b, x_prawdziwe

```

```
#czasy
```

```
def zmierz_czas(funkcja, A, b):  
    start = time.perf_counter()  
    wynik = funkcja(A, b)  
    koniec = time.perf_counter()  
    return wynik, koniec - start
```

PYTHON

```
# przykład a)  
A1 = [  
    [1.0, 2.0, 1.0],  
    [3.0, -7.0, 2.0],  
    [2.0, 4.0, 5.0]  
]  
b1 = [-9.0, 61.0, -9.0]  
  
wynik_Gauss_1, czas_Gauss_1 = zmierz_czas(rozwiaz_uklad_Gaussa, A1, b1)  
print("Gauss, przykład a:")  
wypisz_wektor(wynik_Gauss_1)  
print("Czas:", czas_Gauss_1)  
  
# przykład b), n = 8  
A8, b8 = tworzenie_macierzy_b(8)  
wynik_Gauss_8, czas_Gauss_8 = zmierz_czas(rozwiaz_uklad_Gaussa, A8, b8)  
print("\nGauss, przykład b), n=8:")  
wypisz_wektor(wynik_Gauss_8)  
print("Czas:", czas_Gauss_8)  
  
# przykład b), n = 10  
A10, b10 = tworzenie_macierzy_b(10)  
wynik_Gauss_10, czas_Gauss_10 = zmierz_czas(rozwiaz_uklad_Gaussa, A10, b10)  
print("\nGauss, przykład b), n=10:")  
wypisz_wektor(wynik_Gauss_10)  
print("Czas:", czas_Gauss_10)  
  
# przykład c)  
A_gesta, b_gesta, x_prawdziwe = tworzenie_macierzy_gestej_10()  
wynik_Gauss_gesta, czas_Gauss_gesta = zmierz_czas(rozwiaz_uklad_Gaussa, A_gesta,  
b_gesta)  
print("\nGauss, przykład c):")  
wypisz_wektor(wynik_Gauss_gesta)  
print("Czas:", czas_Gauss_gesta)  
print("Oczekiwane rozwiązanie:", x_prawdziwe)
```

eliminacja Gaussa bez pivotingu

```
def rozwarz_uklad_Gaussa_bez_pivotingu(A, b):  
    n = len(A)  
  
    M = kopiu_j_macierz(A)  
  
    bb = []  
    for x in b:  
        bb.append(float(x))  
  
    eps = 1e-12  
  
    # eliminacja w przód  
    for i in range(n):  
        if abs(M[i][i]) < eps:  
            raise ValueError("Pivot jest zerowy lub bardzo mały. Bez pivotingu nie  
da się bezpiecznie kontynuować.")  
  
        for k in range(i + 1, n):  
            wspolczynnik = M[k][i] / M[i][i]  
  
            for j in range(i, n):  
                M[k][j] = M[k][j] - wspolczynnik * M[i][j]  
  
            bb[k] = bb[k] - wspolczynnik * bb[i]  
  
    # podstawianie w tył  
    x = [0.0] * n  
  
    for i in range(n - 1, -1, -1):  
        suma = 0.0  
  
        for j in range(i + 1, n):  
            suma += M[i][j] * x[j]  
  
        if abs(M[i][i]) < eps:  
            raise ValueError("Dzielenie przez zero w podstawianiu w tył")  
  
        x[i] = (bb[i] - suma) / M[i][i]  
  
    return x
```

Dodatkowe funkcje do porównania wyników


```
def mnozenie_macierzy_przez_wektor(A, x):
    wynik = []

    for i in range(len(A)):
        suma = 0.0

        for j in range(len(A[i])):
            suma += A[i][j] * x[j]

        wynik.append(suma)

    return wynik

def blad_rozwiazania(A, x, b):
    Ax = mnozenie_macierzy_przez_wektor(A, x)

    maksimum = 0.0

    for i in range(len(b)):
        roznica = abs(Ax[i] - b[i])

        if roznica > maksimum:
            maksimum = roznica

    return maksimum

def uruchom_metode(nazwa, funkcja, A, b):
    try:
        wynik, czas = zmierz_czas(funkcja, A, b)

        print(nazwa)
        wypisz_wektor(wynik)
        print("Czas:", czas)
        print("Błąd ||Ax - b|| =", blad_rozwiazania(A, wynik, b))
        print()

    except ValueError as blad:
        print(nazwa)
        print("Metoda nie może być zastosowana:", blad)
        print()
```

testy do obu wersji

```

print("-----ZADANIE 4-----")

# przykład a)
A1 = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]

b1 = [-9.0, 61.0, -9.0]

print("Przykład a):")
uruchom_metode("Gauss bez pivotingu:", rozwiaz_uklad_Gaussa_bez_pivotingu, A1, b1)
uruchom_metode("Gauss z pivotingiem częściowym:",
    rozwiaz_uklad_Gaussa_pivoting_czesciowy, A1, b1)

# przykład b), n = 8
A8, b8 = tworzenie_macierzy_b(8)

print("Przykład b), n = 8:")
uruchom_metode("Gauss bez pivotingu:", rozwiaz_uklad_Gaussa_bez_pivotingu, A8, b8)
uruchom_metode("Gauss z pivotingiem częściowym:",
    rozwiaz_uklad_Gaussa_pivoting_czesciowy, A8, b8)

# przykład b), n = 10
A10, b10 = tworzenie_macierzy_b(10)

print("Przykład b), n = 10:")
uruchom_metode("Gauss bez pivotingu:", rozwiaz_uklad_Gaussa_bez_pivotingu, A10,
    b10)
uruchom_metode("Gauss z pivotingiem częściowym:",
    rozwiaz_uklad_Gaussa_pivoting_czesciowy, A10, b10)

# przykład c)
A_gesta, b_gesta, x_prawdziwe = tworzenie_macierzy_gestej_10()

print("Przykład c):")
uruchom_metode("Gauss bez pivotingu:", rozwiaz_uklad_Gaussa_bez_pivotingu, A_gesta,
    b_gesta)
uruchom_metode("Gauss z pivotingiem częściowym:",
    rozwiaz_uklad_Gaussa_pivoting_czesciowy, A_gesta, b_gesta)

print("Oczekiwane rozwiązanie dla przykładu c):")
print(x_prawdziwe)

```

Lab 6

Używasz `x_stare = x[:]` wtedy, gdy kopiujesz **wektor**, czyli zwykłą listę liczb.

Na przykład w metodzie Gaussa-Seidla:

```
x = [0.0, 0.0, 0.0, 0.0]

x_stare = x[:]
```

PYTHON

To tworzy kopię wektora `x`, żeby zapamiętać poprzednie przybliżenie. Dzięki temu później możesz porównać:

```
x - x_stare
```

PYTHON

czyli różnicę między nowym i starym przybliżeniem.

To samo „normalnie”, bez `[:]`, wygląda tak:

```
x_stare = []

for i in range(len(x)):
    x_stare.append(x[i])
```

PYTHON

Dla **macierzy** nie wystarczy zwykłe:

```
B = A[:]
```

PYTHON

bo macierz jest listą list. Taki zapis kopiuje tylko listę zewnętrzną, ale wiersze dalej są współdzielone.

Dla macierzy trzeba kopiować każdy wiersz osobno:

```
B = []

for i in range(len(A)):
    nowy_wiersz = []

    for j in range(len(A[i])):
        nowy_wiersz.append(A[i][j])

    B.append(nowy_wiersz)
```

PYTHON

Albo krócej:

```
B = []  
  
for i in range(len(A)):  
    B.append(A[i][:])
```

Czyli zasada jest taka:

```
# wektor  
x_stare = x[:]
```

bo `x` jest jedną listą liczb.

```
# macierz  
B = []  
for i in range(len(A)):  
    B.append(A[i][:])
```

bo `A` jest listą wierszy, a każdy wiersz też trzeba skopiować.

W Twoich zadaniach:

- `x_stare = x[:]` dawaj w metodach iteracyjnych, np. Jacobi, Gauss-Seidel, gdy chcesz zapamiętać poprzedni wektor przybliżeń.
- kopię macierzy rób przy eliminacji Gaussa, LU, Gauss-Jordan, gdy zmieniasz macierz w trakcie obliczeń i nie chcesz zepsuć oryginalnej macierzy `A`.

Zadanie 1

Napisz program implementujący metodę Jacobiego iteracyjnego rozwiązywania układów równań liniowych, w której warunkiem zatrzymania będzie:

- liczba iteracji,
- norma wektora powstałego przez odjęcie wektorów określających kolejne przybliżenia,
- błąd uzyskanego przybliżenia.

```

def norma_max(wektor):
    maksimum = abs(wektor[0])
    for i in range(1, len(wektor)):
        if abs(wektor[i]) > maksimum:
            maksimum = abs(wektor[i])
    return maksimum

def odejmij_wektory(wektor1, wektor2):
    wynik = []
    for i in range(len(wektor1)):
        wynik.append(wektor1[i] - wektor2[i])
    return wynik

def wypisz_wektor(wektor, nazwa="x"):
    for i in range(len(wektor)):
        print(f"{nazwa}{i+1} = {wektor[i]}")

def jacobi(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
rozwiązanie_dokładne=None):
    n = len(A)

    # Sprawdzamy, czy macierz A nie jest pusta
    if n == 0:
        raise ValueError("Macierz A nie może być pusta")

    # Sprawdzamy, czy macierz A jest kwadratowa
    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz A musi być kwadratowa")

    # Sprawdzamy, czy wektor b ma dobry rozmiar
    if len(b) != n:
        raise ValueError("Wektor b musi mieć tyle elementów, ile macierz A ma wierszy")

    # Sprawdzamy, czy przybliżenie początkowe x0 ma dobry rozmiar
    if len(x0) != n:
        raise ValueError("Wektor x0 musi mieć tyle elementów, ile jest niewiadomych")

    x_stare = x0[:]
    #      /\
    # normalna wersja tego dla wektora
    # B = []
    #
    # for i in range(len(x0)):
    #     B.append(x0[i])

    # wersja dla macierzy

```

```

# B = []
#
# for i in range(len(A)):
#     nowy_wiersz = []
#     for j in range(len(A[i])):
#         nowy_wiersz.append(A[i][j])
#     B.append(nowy_wiersz)
#-----

for i in range(n):
    if A[i][i] == 0:
        raise ValueError("Na przekątnej macierzy nie może być zera")

for krok in range(1, max_iter + 1):
    x_nowe = [0.0] * n

    for i in range(n):
        suma = 0.0
        for j in range(n):
            if j != i:
                suma += A[i][j] * x_stare[j]

        x_nowe[i] = (b[i] - suma) / A[i][i]

    if warunek_stopu == "iteracje":
        if krok == max_iter:
            return x_nowe, krok

    elif warunek_stopu == "roznica":
        roznica = odejmij_wektory(x_nowe, x_stare)
        norma_roznicy = norma_max(roznica)
        norma_biezaca = norma_max(x_nowe)

        if norma_biezaca == 0:
            if norma_roznicy <= epsilon:
                return x_nowe, krok
        else:
            if (norma_roznicy / norma_biezaca) <= epsilon:
                return x_nowe, krok

    elif warunek_stopu == "blad":
        if rozwiazanie_dokladne is None:
            raise ValueError("Dla warunku 'blad' trzeba podać dokładne rozwiązanie")

        if len(rozwiazanie_dokladne) != n:
            raise ValueError("Rozwiązanie dokładne musi mieć tyle elementów, ile jest niewiadomych")

        blad = odejmij_wektory(x_nowe, rozwiazanie_dokladne)
        if norma_max(blad) <= epsilon:

```

```

        return x_nowe, krok

    else:
        raise ValueError("Nieznany warunek stopu")

    x_stare = x_nowe[:]

    return x_stare, max_iter

A = [
    [4.0, -2.0, 0.0, 0.0],
    [-2.0, 5.0, -1.0, 0.0],
    [0.0, -1.0, 4.0, 2.0],
    [0.0, 0.0, 2.0, 3.0]
]

b = [0.0, 2.0, 3.0, -2.0]
x0 = [0.0, 0.0, 0.0, 0.0]
x_dokladne = [0.5, 1.0, 2.0, -2.0]

print("\nDane:")
print("Macierz A:")
for wiersz in A:
    print(wiersz)

print("\nWektor b:")
print(b)

print("\nPrzybliżenie początkowe x^(0):")
print(x0)

print("\nDokładne rozwiązanie:")
print(x_dokladne)

print("\n----- a) LICZBA ITERACJI -----")
wynik_a, iteracje_a = jacobi(
    A, b, x0,
    max_iter=10,
    warunek_stopu="iteracje"
)

print("\nWynik końcowy po zadanej liczbie iteracji:")
wypisz_wektor(wynik_a)
print("Liczba iteracji:", iteracje_a)

blad_a = odejmij_wektory(wynik_a, x_dokladne)
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a))

print("\n----- b) WARUNEK ZE SLAJDU -----")
wynik_b, iteracje_b = jacobi(
    A, b, x0,
    max_iter=100,
    epsilon=1e-3,

```

```

        warunek_stopu="roznica"
    )

    print("\nWynik końcowy:")
    wypisz_wektor(wynik_b)
    print("Liczba iteracji:", iteracje_b)

    blad_b = odejmij_wektory(wynik_b, x_dokladne)
    print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b))

    print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----")
    print("\n")
    wynik_c, iteracje_c = jacobi(
        A, b, x0,
        max_iter=100,
        epsilon=1e-3,
        warunek_stopu="blad",
        rozwiazanie_dokladne=x_dokladne
    )

    print("\nWynik końcowy:")
    wypisz_wektor(wynik_c)
    print("Liczba iteracji:", iteracje_c)

    blad_c = odejmij_wektory(wynik_c, x_dokladne)
    print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c))

```

do punktu b)

To może oznaczać po prostu:

$$||x^{(k)} - x^{(k-1)}|| \leq$$

A tu jest wersja ze slajdu:

$$\frac{x^{(k)} - x^{(k-1)}}{x^{(k)}} \leq$$

To też jest zgodne z wykładem, bo wykład pokazuje właśnie taki warunek stopu z ilorazem norm.

Jeżeli chcesz mieć **dostłownie punkt b)**, to w gałęzi **"roznica"** daj prostszą wersję:

```

elif warunek_stopu == "roznica":
    roznica = odejmij_wektory(x_nowe, x_stare)
    norma_roznicy = norma_max(roznica)

    if norma_roznicy <= epsilon:
        return x_nowe, krok

```

PYTHON

Zadanie 2

Napisz program implementujący metodę Gaussa-Seidla iteracyjnego rozwiązywania układów równań liniowych z analogicznymi warunkami zatrzymania jak w zadaniu 1.

```

def norma_max(wektor):
    maksimum = abs(wektor[0])
    for i in range(1, len(wektor)):
        if abs(wektor[i]) > maksimum:
            maksimum = abs(wektor[i])
    return maksimum

def odejmij_wektory(wektor1, wektor2):
    wynik = []
    for i in range(len(wektor1)):
        wynik.append(wektor1[i] - wektor2[i])
    return wynik

def wypisz_wektor(wektor, nazwa="x"):
    for i in range(len(wektor)):
        print(f"{nazwa}{i+1} = {wektor[i]}")

def gauss_seidel(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
rozwiązanie_dokładne=None):
    n = len(A)

    if n == 0:
        raise ValueError("Macierz A nie może być pusta")

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz A musi być kwadratowa")

    if len(b) != n:
        raise ValueError("Wektor b musi mieć tyle elementów, ile macierz A ma wierszy")

    if len(x0) != n:
        raise ValueError("Wektor x0 musi mieć tyle elementów, ile jest niewiadomych")

    if epsilon <= 0:
        raise ValueError("Dokładność epsilon musi być dodatnia")

    if max_iter <= 0:
        raise ValueError("Liczba iteracji musi być dodatnia")

    # x = x0[:]

    x = []

    for i in range(len(x0)):
        x.append(x0[i])

    for i in range(n):
        if A[i][i] == 0:
            raise ValueError("Na przekątnej macierzy nie może być zera")

```

```

for krok in range(1, max_iter + 1):
    x_stare = x[:]

    for i in range(n):
        suma1 = 0.0
        for j in range(i):
            suma1 += A[i][j] * x[j]

        suma2 = 0.0
        for j in range(i + 1, n):
            suma2 += A[i][j] * x_stare[j]

        x[i] = (b[i] - suma1 - suma2) / A[i][i]

    if warunek_stopu == "iteracje":
        if krok == max_iter:
            return x, krok

    elif warunek_stopu == "roznica":
        roznica = odejmij_wektory(x, x_stare)
        norma_roznicy = norma_max(roznica)
        norma_biezaca = norma_max(x)

        if norma_biezaca == 0:
            if norma_roznicy <= epsilon:
                return x, krok
        else:
            if (norma_roznicy / norma_biezaca) <= epsilon:
                return x, krok

    elif warunek_stopu == "blad":
        if rozwiazanie_dokladne is None:
            raise ValueError("Dla warunku 'blad' trzeba podać dokładne rozwiązanie")

        if len(rozwiazanie_dokladne) != n:
            raise ValueError("Rozwiązanie dokładne musi mieć tyle elementów, ile jest niewiadomych")

        blad = odejmij_wektory(x, rozwiazanie_dokladne)

        if norma_max(blad) <= epsilon:
            return x, krok

    else:
        raise ValueError("Niepoprawny warunek stopu")

return x, max_iter

A = [
    [4.0, -2.0, 0.0, 0.0],
    [-2.0, 5.0, -1.0, 0.0],

```

```

    [0.0, -1.0, 4.0, 2.0],
    [0.0, 0.0, 2.0, 3.0]
]

b = [0.0, 2.0, 3.0, -2.0]
x0 = [0.0, 0.0, 0.0, 0.0]
x_dokladne = [0.5, 1.0, 2.0, -2.0]

print("\nDane:")
print("Macierz A:")
for wiersz in A:
    print(wiersz)

print("\nWektor b:")
print(b)

print("\nPrzybliżenie początkowe x^(0):")
print(x0)

print("\nDokładne rozwiązanie:")
print(x_dokladne)

print("\n----- a) LICZBA ITERACJI -----")
wynik_a, iteracje_a = gauss_seidel(
    A, b, x0,
    max_iter=10,
    warunek_stopu="iteracje"
)

print("\nWynik końcowy po zadanej liczbie iteracji:")
wypisz_wektor(wynik_a)
print("Liczba iteracji:", iteracje_a)

blad_a = odejmij_wektory(wynik_a, x_dokladne)
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a))

print("\n----- b) NORMA RÓŻNICY KOLEJNYCH PRZYBLIŻEŃ -----")
wynik_b, iteracje_b = gauss_seidel(
    A, b, x0,
    max_iter=100,
    epsilon=1e-3,
    warunek_stopu="roznica"
)

print("\nWynik końcowy:")
wypisz_wektor(wynik_b)
print("Liczba iteracji:", iteracje_b)

blad_b = odejmij_wektory(wynik_b, x_dokladne)
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b))

print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----")

```

```

")
wynik_c, iteracje_c = gauss_seidel(
    A, b, x0,
    max_iter=100,
    epsilon=1e-3,
    warunek_stopu="blad",
    rozwiazanie_dokladne=x_dokladne
)

print("\nWynik końcowy:")
wypisz_wektor(wynik_c)
print("Liczba iteracji:", iteracje_c)

blad_c = odejmij_wektory(wynik_c, x_dokladne)
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c))

```

Zadanie 3

Sprawdź zbieżność układu pod kątem metod z zadania 1 oraz 2.

Układ do testowania:

$$\begin{aligned}
 4x_1 - 2x_2 &= 0, \\
 -2x_1 + 5x_2 - x_3 &= 2, \\
 -x_2 + 4x_3 + 2x_4 &= 3, \\
 2x_3 + 3x_4 &= -2.
 \end{aligned}$$

```
def sprawdz_macierz(A):
    n = len(A)

    if n == 0:
        raise ValueError("Macierz A nie może być pusta")

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz A musi być kwadratowa")

    for i in range(n):
        if A[i][i] == 0:
            raise ValueError("Na przekątnej macierzy A nie może być zera")

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def wypisz_macierz(macierz):
    for wiersz in macierz:
        print(wiersz)

def norma_wierszowa_macierzy(macierz):
    maksimum = 0.0
    for i in range(len(macierz)):
        suma = 0.0
        for j in range(len(macierz[i])):
            suma += abs(macierz[i][j])
        if suma > maksimum:
            maksimum = suma
    return maksimum

def macierz_iteracji_jacobiego(A):
    sprawdz_macierz(A)
    n = len(A)
    W = zeros(n, n)

    for i in range(n):
        for j in range(n):
            if i == j:
                W[i][j] = 0.0
            else:
                W[i][j] = -A[i][j] / A[i][i]

    return W

def rozwiaz_uklad_dolnotrojkatny(LD, b):
```

```

n = len(LD)
x = [0.0] * n

for i in range(n):
    suma = 0.0
    for j in range(i):
        suma += LD[i][j] * x[j]

    if LD[i][i] == 0:
        raise ValueError("Dzielenie przez zero w układzie dolnotrójkątnym")

    x[i] = (b[i] - suma) / LD[i][i]

return x

def macierz_iteracji_gaussa_seidla(A):
    sprawdz_macierz(A)
    n = len(A)

    LD = zeros(n, n)
    U = zeros(n, n)

    for i in range(n):
        for j in range(n):
            if j <= i:
                LD[i][j] = A[i][j]
            else:
                U[i][j] = A[i][j]

    W = zeros(n, n)

    for kolumna in range(n):
        prawa_strona = []
        for i in range(n):
            prawa_strona.append(-U[i][kolumna])

        rozwiazanie = rozwiadz_uklad_dolnotrojkatny(LD, prawa_strona)

        for i in range(n):
            W[i][kolumna] = rozwiazanie[i]

    return W

A = [
    [4.0, -2.0, 0.0, 0.0],
    [-2.0, 5.0, -1.0, 0.0],
    [0.0, -1.0, 4.0, 2.0],
    [0.0, 0.0, 2.0, 3.0]
]

print("\nMacierz A:")
wypisz_macierz(A)

```

```

print("\n----- METODA JACOBIEGO -----")
WJ = macierz_iteracji_jacobiego(A)
print("Macierz iteracyjna W_J:")
wypisz_macierz(WJ)

norma_WJ = norma_wierszowa_macierzy(WJ)
print("Norma wierszowa ||W_J|| =", norma_WJ)

if norma_WJ < 1:
    print("Metoda Jacobiego jest zbieżna, ponieważ ||W_J|| < 1.")
else:
    print("Metoda Jacobiego może nie być zbieżna, ponieważ ||W_J|| >= 1.")

print("\n----- METODA GAUSSA-SEIDLA -----")
WGS = macierz_iteracji_gaussa_seidla(A)
print("Macierz iteracyjna W_GS:")
wypisz_macierz(WGS)

norma_WGS = norma_wierszowa_macierzy(WGS)
print("Norma wierszowa ||W_GS|| =", norma_WGS)

if norma_WGS < 1:
    print("Metoda Gaussa-Seidla jest zbieżna, ponieważ ||W_GS|| < 1.")
else:
    print("Metoda Gaussa-Seidla może nie być zbieżna, ponieważ ||W_GS|| >= 1.")

print("\n----- WNIOSEK KOŃCOWY -----")
print("Zbieżność metod z zadania 1 i 2 badamy przez normę macierzy iteracji.")
print("Dla Jacobiego sprawdzamy macierz W_J.")
print("Dla Gaussa-Seidla sprawdzamy macierz W_GS.")
print("Jeżeli ||W|| < 1, to metoda jest zbieżna.")

```

Lab 7

Zadanie 1

Napisz program implementujący metodę bisekcji. Program przetestuj dla następujących funkcji:

a)

$$f(x) = x^2 - 4, x \in [0, 2.2]$$

b)

$$f(x) = \sin x - \frac{1}{2}, x \in [0, 2.2]$$


```

import math

def sgn(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0

def bisekcja(f, a, b, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje"):
    fa = f(a)
    fb = f(b)

    # Jeżeli pierwiastek jest dokładnie na końcu przedziału
    if fa == 0:
        return a, [[1, a, b, a, fa, 0.0]]

    if fb == 0:
        return b, [[1, a, b, b, fb, 0.0]]

    if sgn(fa) == sgn(fb):
        raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne znaki.")
    # if f(b) * f(a) >= 0:
    #     raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne znaki.")

    a0 = a
    b0 = b
    historia = []

    x1 = a
    x2 = b

    historia.append([1, a, b, x1, f(x1), None])
    historia.append([2, a, b, x2, f(x2), None])

    if warunek_stopu == "iteracje" and max_iter == 1:
        return x1, historia
    if warunek_stopu == "iteracje" and max_iter == 2:
        return x2, historia

    for i in range(3, max_iter + 1):
        miejsce_zerowe = a + ((b - a) / 2.0) # punkt środkowy (a+b)/2

        fa = f(a)
        fc = f(miejsce_zerowe)

        blad = (b0 - a0) / (2 ** (i - 2))

        historia.append([i, a, b, miejsce_zerowe, fc, blad])

```

```

if warunek_stopu == "iteracje":
    if i == max_iter:
        return miejsce_zerowe, historia

    elif warunek_stopu == "blad":
        if blad < epsilon: # alternatywnie |b-a| < epsilon | jak będzie źle to
zrobić <= zamiast <
            return miejsce_zerowe, historia

    elif warunek_stopu == "wartosc":
        if abs(fc) < epsilon: # jak będzie źle to zrobić <= zamiast <
            return miejsce_zerowe, historia

    else:
        raise ValueError("Niepoprawny warunek stopu.")

if fc == 0:
    return miejsce_zerowe, historia

# if fa * fc < 0: # można też tak ale gorsze
if sgn(fa) != sgn(fc):
    b = miejsce_zerowe
else:
    a = miejsce_zerowe

return miejsce_zerowe, historia

def wypisz_historie(historia):
    print("i          a          b          x          f(x)          blad")
    for krok in historia:
        print(
            f"{krok[0]} "
            f"{krok[1]} "
            f"{krok[2]} "
            f"{krok[3]} "
            f"{krok[4]} "
            f"{krok[5]}"
        )

```

punkt a)

```
def f1(x):  
    return x**2 - 4  
  
print("f(x) = x^2 - 4, przedział [0, 2.2]")  
  
wynik_a_iter, historia_a_iter = bisekcja(f1, 0.0, 2.2, max_iter=12,  
warunek_stopu="iteracje")  
print("\nWarunek stopu: liczba iteracji")  
wypisz_historie(historia_a_iter)  
print("Przybliżony pierwiastek:", wynik_a_iter)  
print("f(x) =", f1(wynik_a_iter))  
  
wynik_a_blad, historia_a_blad = bisekcja(f1, 0.0, 2.2, epsilon=1e-3,  
warunek_stopu="blad")  
print("\nWarunek stopu: dostatecznie mały błąd")  
print("Przybliżony pierwiastek:", wynik_a_blad)  
print("f(x) =", f1(wynik_a_blad))  
print("Liczba iteracji:", len(historia_a_blad))  
  
wynik_a_wartosc, historia_a_wartosc = bisekcja(f1, 0.0, 2.2, epsilon=1e-3,  
warunek_stopu="wartosc")  
print("\nWarunek stopu: wartość funkcji bliska zeru")  
print("Przybliżony pierwiastek:", wynik_a_wartosc)  
print("f(x) =", f1(wynik_a_wartosc))  
print("Liczba iteracji:", len(historia_a_wartosc))
```

$f(x) = x^2 - 4$, przedział $[0, 2.2]$

Warunek stopu: liczba iteracji

i	a	b	x	f(x)	blad
1	0.0	2.2	0.0	-4.0	None
2	0.0	2.2	2.2	0.8400000000000007	None
3	0.0	2.2	1.1	-2.79	1.1
4	1.1	2.2	1.6500000000000001	-1.2774999999999994	0.55
5	1.6500000000000001	2.2	1.9250000000000003	-0.29437499999999917	0.275
6	1.9250000000000003	2.2	2.0625	0.25390625	0.1375
7	1.9250000000000003	2.0625	1.9937500000000001	-0.024960937499999503	0.06875
8	1.9937500000000001	2.0625	2.028125	0.1132910156250011	0.034375
9	1.9937500000000001	2.028125	2.0109375000000003	0.04386962890625146	0.0171875
10	1.9937500000000001	2.0109375000000003	2.00234375	0.009380493164062642	0.00859375
11	1.9937500000000001	2.00234375	1.998046875	-0.007808685302734375	0.004296875
12	1.998046875	2.00234375	2.0001953125	0.0007812881469719812	0.0021484375

Przybliżony pierwiastek: 2.0001953125
 $f(x) = 0.0007812881469719812$

Warunek stopu: dostatecznie mały błąd

Przybliżony pierwiastek: 1.9996582031249999

$f(x) = -0.0013670706748967199$

Liczba iteracji: 14

Warunek stopu: wartość funkcji bliska zeru

Przybliżony pierwiastek: 2.0001953125

$f(x) = 0.0007812881469719812$

Liczba iteracji: 12

punkt b)

```
def f2(x):  
    return math.sin(x) - 0.5  
  
print("f(x) = sin(x) - 1/2, przedział [0, 2.2]")  
  
wynik_b_iter, historia_b_iter = bisekcja(f2, 0.0, 2.2, max_iter=12,  
warunek_stopu="iteracje")  
print("\nWarunek stopu: liczba iteracji")  
wypisz_historie(historia_b_iter)  
print("Przybliżony pierwiastek:", wynik_b_iter)  
print("f(x) =", f2(wynik_b_iter))  
  
wynik_b_blad, historia_b_blad = bisekcja(f2, 0.0, 2.2, epsilon=1e-3,  
warunek_stopu="blad")  
print("\nWarunek stopu: dostatecznie mały błąd")  
print("Przybliżony pierwiastek:", wynik_b_blad)  
print("f(x) =", f2(wynik_b_blad))  
print("Liczba iteracji:", len(historia_b_blad))  
  
wynik_b_wartosc, historia_b_wartosc = bisekcja(f2, 0.0, 2.2, epsilon=1e-3,  
warunek_stopu="wartosc")  
print("\nWarunek stopu: wartość funkcji bliska zeru")  
print("Przybliżony pierwiastek:", wynik_b_wartosc)  
print("f(x) =", f2(wynik_b_wartosc))  
print("Liczba iteracji:", len(historia_b_wartosc))
```

$f(x) = \sin(x) - 1/2$, przedział $[0, 2.2]$

Warunek stopu: liczba iteracji

i	a	b	x	f(x)	blad
1	0.0	2.2	0.0	-0.5	None
2	0.0	2.2	2.2	0.3084964038195901	None
3	0.0	2.2	1.1	0.3912073600614354	1.1
4	0.0	1.1	0.55	0.02268722893065922	0.55
5	0.0	0.55	0.275	-0.22845306304388713	0.275
6	0.275	0.55	0.4125000000000003	-0.09909911800037147	0.1375
7	0.4125000000000003	0.55	0.4812500000000007	-0.03711244185871215	0.06875
8	0.4812500000000007	0.55	0.515625	-0.006921314246076948	0.034375
9	0.515625	0.55	0.5328125	0.007957983468939722	0.0171875
10	0.515625	0.5328125	0.52421875	0.0005368174551121374	0.00859375
11	0.515625	0.52421875	0.519921875	-0.00318766204596449	0.004296875
12	0.519921875	0.52421875	0.5220703124999999	-0.0013242714062212668	0.0021484375

Przybliżony pierwiastek: 0.5220703124999999
 $f(x) = -0.0013242714062212668$

Warunek stopu: dostatecznie mały błąd

Przybliżony pierwiastek: 0.523681640625

$f(x) = 7.176150147314431e-05$

Liczba iteracji: 14

Warunek stopu: wartość funkcji bliska zeru

Przybliżony pierwiastek: 0.52421875

$f(x) = 0.0005368174551121374$

Liczba iteracji: 10

Zadanie 2

Napisz program implementujący metodę Newtona (czyli metodę stycznych). Program przetestuj dla funkcji z zadania 1.

Rozważane funkcje:

a)

$$f(x) = x^2 - 4, \quad x \in 0, 2.2 >$$

b)

$$f(x) = \sin x - \frac{1}{2}, \quad x \in 0, 2.2 >$$

```

import math

def newton(f, a, b, df, ddf, max_iter=100, epsilon=1e-3):
    lista_iteracji = []

    fa = f(a)
    fb = f(b)

    if fa == 0:
        return a, a, [[0, a, fa, df(a), 0.0, a]]

    if fb == 0:
        return b, b, [[0, b, fb, df(b), 0.0, b]]

    if fa * fb >= 0:
        raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne znaki.")

    c = a + (b - a) / 2.0
    iloczyn_pochodnych = df(c) * ddf(c)

    x = 0.0

    if iloczyn_pochodnych < 0:
        x = a
    elif iloczyn_pochodnych > 0:
        x = b
    else:
        raise ValueError("Nie można jednoznacznie wybrać punktu startowego, bo  $f'(c) * f''(c) = 0$ .")

    punkt_startowy = x

    for i in range(1, max_iter+1):
        fx = f(x)
        dfx = df(x)

        if dfx == 0: # lub abs(dfx) < 1e-12
            raise ValueError("Pochodna  $f'(x) = 0$ , metoda Newtona nie może wykonać kolejnego kroku.")

        #  $h = f(x) / f'(x)$ 
        h = fx / dfx

        # wzór Newtona:  $x_{nowe} = x - f(x) / f'(x)$ 
        x_nowe = x - h

        lista_iteracji.append([i, x, fx, dfx, h, x_nowe])

    if abs(h) < epsilon: # lub zamiast < zrobić <=
        return punkt_startowy, x_nowe, lista_iteracji

```

```

        x = x_nowe

    return punkt_startowy, x, lista_iteracji

def wypisz_historie(historia):
    print("i          x          f(x)          f'(x)          h
x_nowe")
    for krok in historia:
        print(
            f"{krok[0]:<2} "
            f"{krok[1]:>14.10f} "
            f"{krok[2]:>14.10f} "
            f"{krok[3]:>14.10f} "
            f"{krok[4]:>14.10f} "
            f"{krok[5]:>14.10f}"
        )

```

punkt a)

PYTHON

```

def f1(x):
    return x**2 - 4

def df1(x):
    return 2*x

def ddf1(x):
    return 2

print("f(x) = x^2 - 4, przedział [0, 2.2]")

punkt_startowy_a, wynik_a, historia_a = newton(f1, 0.0, 2.2, df1, ddf1,
max_iter=100, epsilon=1e-3)
print("Punkt startowy x0 =", punkt_startowy_a)
wypisz_historie(historia_a)
print("Przybliżony pierwiastek:", wynik_a)
print("f(x) =", f1(wynik_a))
print("Liczba iteracji:", len(historia_a))

```

f(x) = x² - 4, przedział [0, 2.2]

Punkt startowy x0 = 2.2

i	x	f(x)	f'(x)	h	x_nowe
1	2.2000000000	0.8400000000	4.4000000000	0.1909090909	2.0090909091
2	2.0090909091	0.0364462810	4.0181818182	0.0090703414	2.0000205677
3	2.0000205677	0.0000822711	4.0000411353	0.0000205676	2.0000000001

Przybliżony pierwiastek: 2.0000000001057563

f(x) = 4.2302517044845445e-10

Liczba iteracji: 3

punkt b)


```

def f2(x):
    return math.sin(x) - 0.5

def df2(x):
    return math.cos(x)

def ddf2(x):
    return -math.sin(x)

print("f(x) = sin(x) - 1/2, przedział [0, 2.2]")

punkt_startowy_b, wynik_b, historia_b = newton(f2, 0.0, 2.2, df2, ddf2,
max_iter=100, epsilon=1e-3)
print("Punkt startowy x0 =", punkt_startowy_b)
wypisz_historie(historia_b)
print("Przybliżony pierwiastek:", wynik_b)
print("f(x) =", f2(wynik_b))
print("Liczba iteracji:", len(historia_b))

```

f(x) = sin(x) - 1/2, przedział [0, 2.2]

Punkt startowy x0 = 0.0

i	x	f(x)	f'(x)	h	x_nowe
1	0.0000000000	-0.5000000000	1.0000000000	-0.5000000000	0.5000000000
2	0.5000000000	-0.0205744614	0.8775825619	-0.0234444738	0.5234444738
3	0.5234444738	-0.0001336352	0.8661025444	-0.0001542949	0.5235987687

Przybliżony pierwiastek: 0.5235987687270579

f(x) = -5.95066923514409e-09

Liczba iteracji: 3

Zadanie 3

Napisz program implementujący metodę siecznych. Program przetestuj dla funkcji z zadania 1.

Rozważane funkcje:

a)

$$f(x) = x^2 - 4, \quad x \in 0, 2.2 >$$

b)

$$f(x) = \sin x - \frac{1}{2}, \quad x \in 0, 2.2 >$$

```

import math

def sieczne(f, a, b, df, ddf, max_iter=100, epsilon=1e-3):
    lista_iteracji = []

    if f(a) * f(b) >= 0:
        raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne znaki.")

    c = (a+b) / 2.0
    iloczyn_pochodnych = df(c) * ddf(c)

    if iloczyn_pochodnych < 0:
        x0 = a
        x1 = b
    elif iloczyn_pochodnych > 0:
        x0 = b
        x1 = a
    else:
        raise ValueError("Nie można jednoznacznie wybrać punktu startowego, bo f'(c) * f''(c) = 0.")

    x0_startowy = x0
    x1_startowy = x1

    for i in range(1, max_iter+1):
        fx0 = f(x0)
        fx1 = f(x1)

        if fx1 - fx0 == 0: # lub if abs(fx1 - fx0) < 1e-12:
            raise ValueError("Mianownik jest równy zero, metoda siecznych nie może wykonać kolejnego kroku.")

        x_nowe = x1 - fx1 * ((x1 - x0) / (fx1 - fx0))

        lista_iteracji.append([i, x0, x1, fx0, fx1, x_nowe])

        if abs(x_nowe - x1) < epsilon: # lub if abs(x_nowe - x1) <= epsilon:
            return x0_startowy, x1_startowy, x0, x_nowe, lista_iteracji

        x0 = x1
        x1 = x_nowe

    return x0_startowy, x1_startowy, x0, x1, lista_iteracji

def wypisz_historie(historia):
    print("i          x0          x1          f(x0)          f(x1)          x_nowe")
    for krok in historia:
        print(
            f"{krok[0]:<2} "
            f"{krok[1]:>14.10f} "

```

```

        f"{krok[2]:>14.10f} "
        f"{krok[3]:>14.10f} "
        f"{krok[4]:>14.10f} "
        f"{krok[5]:>14.10f}"
    )

```

punkt a)

PYTHON

```

def f1(x):
    return x**2 - 4

def df1(x):
    return 2*x

def ddf1(x):
    return 2

print("f(x) = x^2 - 4, przedział [0, 2.2]")

x0a, x1a, wynik0_a, wynik1_a, historia_a = sieczne(f1, 0.0, 2.2, df1, ddf1,
max_iter=100, epsilon=1e-3)
print("Punkty startowe: x0 =", x0a, ", x1 =", x1a)
wypisz_historie(historia_a)
print("Ostatnie przybliżenia:", wynik0_a, wynik1_a)
print("Przybliżony pierwiastek:", wynik1_a)
print("f(x) =", f1(wynik1_a))
print("Liczba iteracji:", len(historia_a))

```

f(x) = x^2 - 4, przedział [0, 2.2]

Punkty startowe: x0 = 2.2 , x1 = 0.0

i	x0	x1	f(x0)	f(x1)	x_nowe
1	2.2000000000	0.0000000000	0.8400000000	-4.0000000000	1.8181818182
2	0.0000000000	1.8181818182	-4.0000000000	-0.6942148760	2.2000000000
3	1.8181818182	2.2000000000	-0.6942148760	0.8400000000	1.9909502262
4	2.2000000000	1.9909502262	0.8400000000	-0.0361171966	1.9995681278
5	1.9909502262	1.9995681278	-0.0361171966	-0.0017273021	2.0000009794

Ostatnie przybliżenia: 1.990950226244344 2.000000979407948

Przybliżony pierwiastek: 2.000000979407948

f(x) = 3.917632751537781e-06

Liczba iteracji: 5

punkt b)

```

def f2(x):
    return math.sin(x) - 0.5

def df2(x):
    return math.cos(x)

def ddf2(x):
    return -math.sin(x)

print("f(x) = sin(x) - 1/2, przedział [0, 2.2]")

x0b, x1b, wynik0_b, wynik1_b, historia_b = sieczne(f2, 0.0, 2.2, df2, ddf2,
max_iter=100, epsilon=1e-3)
print("Punkty startowe: x0 =", x0b, ", x1 =", x1b)
wypisz_historie(historia_b)
print("Ostatnie przybliżenia:", wynik0_b, wynik1_b)
print("Przybliżony pierwiastek:", wynik1_b)
print("f(x) =", f2(wynik1_b))
print("Liczba iteracji:", len(historia_b))

```

f(x) = sin(x) - 1/2, przedział [0, 2.2]

Punkty startowe: x0 = 0.0 , x1 = 2.2

i	x0	x1	f(x0)	f(x1)	x_nowe
1	0.0000000000	2.2000000000	-0.5000000000	0.3084964038	1.3605502694
2	2.2000000000	1.3605502694	0.3084964038	0.4779795920	3.7279817771
3	1.3605502694	3.7279817771	0.4779795920	-1.0533569783	2.0995020897
4	3.7279817771	2.0995020897	-1.0533569783	0.3634606277	2.5172610496
5	2.0995020897	2.5172610496	0.3634606277	0.0845550978	2.6439119973
6	2.5172610496	2.6439119973	0.0845550978	-0.0226111647	2.6171897307
7	2.6439119973	2.6171897307	-0.0226111647	0.0006962502	2.6179879910

Ostatnie przybliżenia: 2.643911997346381 2.61798799102743

Przybliżony pierwiastek: 2.61798799102743

f(x) = 5.098251766644246e-06

Liczba iteracji: 7

Funkcje do automatycznego liczenia pochodnych

PYTHON

```
import math

def pochodna(f, x, rzad=1, h=1e-5):
    if rzad < 0:
        raise ValueError("Rząd pochodnej nie może być ujemny.")

    if rzad == 0:
        return f(x)

    if h <= 0:
        raise ValueError("Krok h musi być dodatni.")

    # Pierwsza pochodna liczona wzorem centralnym:
    #  $f'(x) \approx (f(x+h) - f(x-h)) / (2h)$ 
    if rzad == 1:
        return (f(x + h) - f(x - h)) / (2 * h)

    # Dla wyższych rzędów liczymy pochodną pochodnej rekurencyjnie.
    # Czyli np. druga pochodna = pochodna z pierwszej pochodnej.
    def poprzednia_pochodna(t):
        return pochodna(f, t, rzad - 1, h)

    return (poprzednia_pochodna(x + h) - poprzednia_pochodna(x - h)) / (2 * h)

def df_num(f):
    return lambda x: pochodna(f, x, rzad=1)

def ddf_num(f):
    return lambda x: pochodna(f, x, rzad=2)

def dddf_num(f):
    return lambda x: pochodna(f, x, rzad=3)

# -----
# PRZYKŁADY TESTOWE
# -----

def f1(x):
    return x**2 - 4

def f2(x):
    return math.sin(x) - 0.5
```

```

def f3(x):
    return x**3 - 3*x**2 - 2*x + 5

print("----- TEST POCHODNYCH -----")

x = 2.0

print("\nFunkcja f1(x) = x^2 - 4")
print("f1(x) =", f1(x))
print("f1'(x) =", pochodna(f1, x, rzad=1))
print("f1''(x) =", pochodna(f1, x, rzad=2))
print("f1'''(x) =", pochodna(f1, x, rzad=3))

x = 0.5

print("\nFunkcja f2(x) = sin(x) - 1/2")
print("f2(x) =", f2(x))
print("f2'(x) =", pochodna(f2, x, rzad=1))
print("f2''(x) =", pochodna(f2, x, rzad=2))
print("f2'''(x) =", pochodna(f2, x, rzad=3))

x = 1.0

print("\nFunkcja f3(x) = x^3 - 3x^2 - 2x + 5")
print("f3(x) =", f3(x))
print("f3'(x) =", pochodna(f3, x, rzad=1))
print("f3''(x) =", pochodna(f3, x, rzad=2))
print("f3'''(x) =", pochodna(f3, x, rzad=3))

# -----
# PRZYKŁAD UŻYCIA W METODZIE NEWTONA LUB SIECZNYCH
# -----

df1 = df_num(f1)
ddf1 = ddf_num(f1)

print("\nPochodne jako funkcje:")
print("df1(2.0) =", df1(2.0))
print("ddf1(2.0) =", ddf1(2.0))

```

Lab 8

Zadanie 1

Napisz program implementujący rozwinięcie funkcji eksponencjalnej w szereg Maclaurina. Porównaj wyniki oraz czas wykonania z funkcją biblioteczną.

```

import time, math

def exp_maclaurin(x, n):
    suma = 1.0
    wyraz = 1.0

    for k in range(1, n+1):
        wyraz = wyraz * (x / k)
        suma += wyraz

    return suma

def porownaj_exp(x, n, liczba_powtorzen=100000):
    start = time.perf_counter()
    for _ in range(liczba_powtorzen):
        wynik_maclaurin = exp_maclaurin(x, n)
    czas_maclaurin = time.perf_counter() - start

    start = time.perf_counter()
    for _ in range(liczba_powtorzen):
        wynik_biblioteczny = math.exp(x)
    czas_biblioteczny = time.perf_counter() - start

    blad = abs(wynik_maclaurin - wynik_biblioteczny)

    print(f"x = {x}, n = {n}")
    print(f"Wynik Maclaurina: {wynik_maclaurin}")
    print(f"Wynik biblioteczny: {wynik_biblioteczny}")
    print(f"Błąd bezwzględny: {blad}")
    print(f"Czas Maclaurina: {czas_maclaurin:.10f} s")
    print(f"Czas biblioteczny: {czas_biblioteczny:.10f} s")
    print("-" * 50)

print("----- ZADANIE 1 -----")

porownaj_exp(1.0, 10)
porownaj_exp(2.0, 15)
porownaj_exp(-1.0, 15)
porownaj_exp(5.0, 25)

```

```
x = 1.0, n = 10
Wynik Maclaurina: 2.7182818011463845
Wynik biblioteczny: 2.718281828459045
Błąd bezwzględny: 2.7312660577649694e-08
Czas Maclaurina: 0.0412331000 s
Czas biblioteczny: 0.0029771002 s
```

```
-----
x = 2.0, n = 15
Wynik Maclaurina: 7.389056095384136
Wynik biblioteczny: 7.38905609893065
Błąd bezwzględny: 3.546514193430994e-09
Czas Maclaurina: 0.0566379000 s
Czas biblioteczny: 0.0032021003 s
```

```
-----
x = -1.0, n = 15
Wynik Maclaurina: 0.3678794411713973
Wynik biblioteczny: 0.36787944117144233
Błąd bezwzględny: 4.50195436485501e-14
Czas Maclaurina: 0.0574267004 s
Czas biblioteczny: 0.0030766996 s
```

```
-----
x = 5.0, n = 25
Wynik Maclaurina: 148.41315909805007
Wynik biblioteczny: 148.4131591025766
Błąd bezwzględny: 4.5265267090144334e-09
Czas Maclaurina: 0.0857619001 s
Czas biblioteczny: 0.0030656001 s
```

Zadanie 2

Napisz program implementujący rozwinięcie funkcji sinus w szereg Maclaurina. Porównaj wyniki oraz czas wykonania z funkcją biblioteczną.


```

import math, time

def sin_maclaurin(x, n):
    x = x % (2 * math.pi)

    suma = x
    wyraz = x
    znak = -1.0

    for k in range(2, n + 1):
        wyraz = wyraz * x * x / ((2 * k - 2) * (2 * k - 1))
        suma += znak * wyraz
        znak = -znak

    return suma

def porownaj_sin(x, n, liczba_powtorzen=100000):
    start = time.perf_counter()
    for _ in range(liczba_powtorzen):
        wynik_maclaurin = sin_maclaurin(x, n)
    czas_maclaurin = time.perf_counter() - start

    start = time.perf_counter()
    for _ in range(liczba_powtorzen):
        wynik_biblioteczny = math.sin(x)
    czas_biblioteczny = time.perf_counter() - start

    blad = abs(wynik_maclaurin - wynik_biblioteczny)

    print(f"x = {x}, n = {n}")
    print(f"Wynik Maclaurina: {wynik_maclaurin}")
    print(f"Wynik biblioteczny: {wynik_biblioteczny}")
    print(f"Błąd bezwzględny: {blad}")
    print(f"Czas Maclaurina: {czas_maclaurin:.10f} s")
    print(f"Czas biblioteczny: {czas_biblioteczny:.10f} s")
    print("-" * 50)

print("----- ZADANIE 2 -----")

porownaj_sin(0.5, 10)
porownaj_sin(1.0, 10)
porownaj_sin(math.pi / 2, 12)
porownaj_sin(10.0, 15)

```

```

x = 0.5, n = 10
Wynik Maclaurina: 0.479425538604203
Wynik biblioteczny: 0.479425538604203
Błąd bezwzględny: 0.0
Czas Maclaurina: 0.0750450008 s
Czas biblioteczny: 0.0029694000 s
-----

x = 1.0, n = 10
Wynik Maclaurina: 0.8414709848078965
Wynik biblioteczny: 0.8414709848078965
Błąd bezwzględny: 0.0
Czas Maclaurina: 0.0743089002 s
Czas biblioteczny: 0.0030261995 s
-----

x = 1.5707963267948966, n = 12
Wynik Maclaurina: 1.0000000000000002
Wynik biblioteczny: 1.0
Błąd bezwzględny: 2.220446049250313e-16
Czas Maclaurina: 0.0896358993 s
Czas biblioteczny: 0.0032490995 s
-----

x = 10.0, n = 15
Wynik Maclaurina: -0.5440211108893696
Wynik biblioteczny: -0.5440211108893698
Błąd bezwzględny: 2.220446049250313e-16
Czas Maclaurina: 0.1106032999 s
Czas biblioteczny: 0.0029934002 s

```

Zadanie 3

Napisz funkcję wyznaczającą współczynniki wielomianu interpolacyjnego Newtona dla stabilizowanych wartości pewnej funkcji. (Interpolacja Newtona)

```
def wspolczynniki_newtona(x, y):
    n = len(x)

    if len(y) != n:
        raise ValueError("Listy x i y muszą mieć taką samą długość.")

    a = y.copy()

    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            if x[i] == x[i - j]:
                raise ValueError("Wartości x muszą być różne.")
            a[i] = (a[i] - a[i - 1]) / (x[i] - x[i - j])

    return a

print("----- ZADANIE 3 -----")

x = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 2.0, 0.0, 5.0]

a = wspolczynniki_newtona(x, y)

print("Węzły x:", x)
print("Wartości y:", y)
print("Współczynniki wielomianu Newtona:")
for i in range(len(a)):
    print(f"a{i} = {a[i]}")
```

```
Węzły x: [0.0, 1.0, 2.0, 3.0]
Wartości y: [1.0, 2.0, 0.0, 5.0]
Współczynniki wielomianu Newtona:
a0 = 1.0
a1 = 1.0
a2 = -1.5
a3 = 1.6666666666666667
```

Zadanie 4

Napisz funkcję umożliwiającą obliczanie wartości wielomianu interpolacyjnego Newtona. Zastosuj algorytm analogiczny do schematu Hornera.

```

def wspolczynniki_newtona(x, y):
    n = len(x)

    if len(y) != n:
        raise ValueError("Listy x i y muszą mieć taką samą długość.")

    a = y.copy()

    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            if x[i] == x[i - j]:
                raise ValueError("Wartości x muszą być różne.")
            a[i] = (a[i] - a[i - 1]) / (x[i] - x[i - j])

    return a

def wartosc_wielomianu_newtona(x_wezly, a, X):
    n = len(a)

    if len(x_wezly) != n:
        raise ValueError("Liczba węzłów x i liczba współczynników musi być taka sama.")

    wynik = a[n - 1]

    for i in range(n - 2, -1, -1):
        wynik = wynik * (X - x_wezly[i]) + a[i]

    return wynik

print("----- ZADANIE 4 -----")

x = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 2.0, 0.0, 5.0]

a = wspolczynniki_newtona(x, y)

X = 1.5
wartosc = wartosc_wielomianu_newtona(x, a, X)

print("Węzły x:", x)
print("Wartości y:", y)
print("Współczynniki Newtona:", a)
print("Punkt X:", X)
print("Wartość wielomianu w punkcie X:", wartosc)

```

Węzły x: [0.0, 1.0, 2.0, 3.0]
Wartości y: [1.0, 2.0, 0.0, 5.0]
Współczynniki Newtona: [1.0, 1.0, -1.5, 1.6666666666666667]
Punkt X: 1.5
Wartość wielomianu w punkcie X: 0.7499999999999999

Zadanie 5

Rozwiązania dwóch poprzednich zadań wykorzystaj do narysowania wykresu wielomianu interpolacyjnego (zaznaczając również punktami zaobserwowane wartości funkcji).

```

import matplotlib.pyplot as plt

def wspolczynniki_newtona(x, y):
    n = len(x)

    if len(y) != n:
        raise ValueError("Listy x i y muszą mieć taką samą długość.")

    a = y.copy()

    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            if x[i] == x[i - j]:
                raise ValueError("Wartości x muszą być różne.")
            a[i] = (a[i] - a[i - j]) / (x[i] - x[i - j])

    return a

def wartosc_wielomianu_newtona(x_wezly, a, X):
    n = len(a)

    if len(x_wezly) != n:
        raise ValueError("Liczba węzłów x i liczba współczynników musi być taka sama.")

    wynik = a[n - 1]

    for i in range(n - 2, -1, -1):
        wynik = wynik * (X - x_wezly[i]) + a[i]

    return wynik

print("----- ZADANIE 5 -----")

x = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 2.0, 0.0, 5.0]

a = wspolczynniki_newtona(x, y)

x_min = min(x)
x_max = max(x)

x_wykres = []
y_wykres = []

liczba_punktow = 200

for i in range(liczba_punktow + 1):
    X = x_min + (x_max - x_min) * i / liczba_punktow
    Y = wartosc_wielomianu_newtona(x, a, X)

```

```
x_wykres.append(X)
y_wykres.append(Y)

plt.plot(x_wykres, y_wykres, label="Wielomian interpolacyjny Newtona")
plt.scatter(x, y, label="Punkty wejściowe")

plt.title("Interpolacja Newtona")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

Lab 9

Zadanie 1

Napisz program implementujący aproksymację średniokwadratową liniową dla zbioru punktów

$$\{(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)\}.$$

```
def aproksymacja_sredniokwadratowa(punkty):
    n = len(punkty)

    A = 0.0
    B = 0.0
    C = 0.0
    D = 0.0

    for x, y in punkty:
        A += x * y
        B += x
        C += y
        D += x * x

    a = (n * A - B * C) / (n * D - B * B)
    b = (C * D - A * B) / (n * D - B * B)

    h = 0.0
    for x, y in punkty:
        h += (a * x + b - y) ** 2

    return a, b, h

punkty = [(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)]

a, b, h = aproksymacja_sredniokwadratowa(punkty)

print("a: ", a, ", b: ", b)
print("Suma kwadratów błędów:", h)
print("Funkcja aproksymująca: ")
print("F(x) = ", a, " * x +", b)
```

```
a: 0.9868421052631559 ,b: 0.9776315789473676
Suma kwadratów błędów: 0.17447368421052628
Funkcja aproksymująca:
F(x) = 0.9868421052631559 * x + 0.9776315789473676
```

Zadanie 2

Napisz program implementujący aproksymację średniokwadratową wielomianem drugiego stopnia dla zbioru punktów

$$\{(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91)\}.$$

Wersja uniwersalna i z pivotem


```

def rozwarz_uklad_gaussa(macierz, wyrazy_wolne):
    n = len(wyrazy_wolne)

    for i in range(n):
        max_wiersz = i

        for k in range(i + 1, n):
            if abs(macierz[k][i]) > abs(macierz[max_wiersz][i]):
                max_wiersz = k

        if abs(macierz[max_wiersz][i]) < 1e-12:
            raise ValueError("Układ nie ma jednoznacznego rozwiązania")

        if max_wiersz != i:
            macierz[i], macierz[max_wiersz] = macierz[max_wiersz], macierz[i]
            wyrazy_wolne[i], wyrazy_wolne[max_wiersz] = wyrazy_wolne[max_wiersz],
            wyrazy_wolne[i]

        for j in range(i + 1, n):
            wspolczynnik = macierz[j][i] / macierz[i][i]

            for k in range(i, n):
                macierz[j][k] = macierz[j][k] - wspolczynnik * macierz[i][k]

            wyrazy_wolne[j] = wyrazy_wolne[j] - wspolczynnik * wyrazy_wolne[i]

    rozwiazanie = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0

        for j in range(i + 1, n):
            suma += macierz[i][j] * rozwiazanie[j]

        rozwiazanie[i] = (wyrazy_wolne[i] - suma) / macierz[i][i]

    return rozwiazanie

def wartosc_wielomianu(wspolczynniki, x):
    wynik = 0.0

    for i in range(len(wspolczynniki)):
        wynik += wspolczynniki[i] * (x ** i)

    return wynik

def aproksymacja_wielomianowa(punkty, stopien):
    n = len(punkty)

    if stopien < 0:

```

```

        raise ValueError("Stopień wielomianu nie może być ujemny")

    if n < stopien + 1:
        raise ValueError("Liczba punktów musi być większa od stopnia wielomianu")

    rozmiar = stopien + 1

    macierz_ukladu = []
    wyrazy_wolne = []

    for i in range(rozmiar):
        wiersz = []

        for j in range(rozmiar):
            suma = 0.0

            for x, y in punkty:
                suma += x ** (i + j)

            wiersz.append(suma)

        macierz_ukladu.append(wiersz)

        suma_prawa = 0.0
        for x, y in punkty:
            suma_prawa += y * (x ** i)

        wyrazy_wolne.append(suma_prawa)

    macierz_kopia = []
    for wiersz in macierz_ukladu:
        nowy_wiersz = []
        for element in wiersz:
            nowy_wiersz.append(element)
        macierz_kopia.append(nowy_wiersz)

    wyrazy_wolne_kopia = []
    for element in wyrazy_wolne:
        wyrazy_wolne_kopia.append(element)

    wspolczynniki = rozwiaz_uklad_gaussa(macierz_kopia, wyrazy_wolne_kopia)

    h = 0.0
    for x, y in punkty:
        h += (wartosc_wielomianu(wspolczynniki, x) - y) ** 2

    return wspolczynniki, h

def wypisz_wielomian(wspolczynniki):
    tekst = "F(x) = "

    for i in range(len(wspolczynniki)):

```

```

        if i == 0:
            tekst += str(wspolczynniki[i])
        elif i == 1:
            tekst += " + " + str(wspolczynniki[i]) + " * x"
        else:
            tekst += " + " + str(wspolczynniki[i]) + " * x^" + str(i)

    print(tekst)

punkty = [
    (0.0, 2.0),
    (0.5, 2.48),
    (1.0, 2.84),
    (1.5, 3.0),
    (2.0, 2.91)
]

stopien = 2

wspolczynniki, h = aproksymacja_wielomianowa(punkty, stopien)

print("Współczynniki:")
for i in range(len(wspolczynniki)):
    print("a" + str(i), "=", wspolczynniki[i])

print("\nWielomian aproksymacyjny:")
wypisz_wielomian(wspolczynniki)

print("\nSuma kwadratów błędów:")
print(h)

```

Współczynniki:

```

a0 = 1.9865714285714295
a1 = 1.2337142857142822
a2 = -0.3828571428571412

```

Wielomian aproksymacyjny:

$$F(x) = 1.9865714285714295 + 1.2337142857142822 * x + -0.3828571428571412 * x^2$$

Suma kwadratów błędów:

```
0.0017028571428571394
```

Wersja prostrza

```

punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)]

def rozwarz_uklad_gaussa(macierz, wyrazy_wolne):
    n = len(wyrazy_wolne)

    for i in range(n):
        if macierz[i][i] == 0:
            raise ValueError("Na przekątnej pojawiło się zero - nie można wykonać eliminacji Gaussa.")

        for j in range(i + 1, n):
            wspolczynnik = macierz[j][i] / macierz[i][i]

            for k in range(i, n):
                macierz[j][k] = macierz[j][k] - wspolczynnik * macierz[i][k]

            wyrazy_wolne[j] = wyrazy_wolne[j] - wspolczynnik * wyrazy_wolne[i]

    rozwiazanie = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0
        for j in range(i + 1, n):
            suma += macierz[i][j] * rozwiazanie[j]

        rozwiazanie[i] = (wyrazy_wolne[i] - suma) / macierz[i][i]

    return rozwiazanie

n = len(punkty)

suma_x = 0.0
suma_x2 = 0.0
suma_x3 = 0.0
suma_x4 = 0.0
suma_y = 0.0
suma_xy = 0.0
suma_x2y = 0.0

for x, y in punkty:
    suma_x += x
    suma_x2 += x ** 2
    suma_x3 += x ** 3
    suma_x4 += x ** 4
    suma_y += y
    suma_xy += x * y
    suma_x2y += x ** 2 * y

macierz_ukladu = [
    [suma_x4, suma_x3, suma_x2],
    [suma_x3, suma_x2, suma_x],
    [suma_x2, suma_x, n]

```

```

]

wyrazy_wolne = [suma_x2y, suma_xy, suma_y]

macierz_kopia = []
for wiersz in macierz_ukladu:
    macierz_kopia.append(wiersz.copy())

wyrazy_wolne_kopia = wyrazy_wolne.copy()

rozwiazanie = rozwarz_uklad_gaussa(macierz_kopia, wyrazy_wolne_kopia)

a = rozwiazanie[0]
b = rozwiazanie[1]
c = rozwiazanie[2]

print("Współczynniki wielomianu aproksymacyjnego:")
print("a =", a)
print("b =", b)
print("c =", c)

print("\nWielomian aproksymacyjny:")
print(f"y = {a} * x^2 + {b} * x + {c}")

h = 0.0
for x, y in punkty:
    h += (a * x**2 + b * x + c - y) ** 2

print("\nSuma kwadratów błędów:")
print(h)

```

Zadanie 3

Napisz funkcję, która dla zadanego zbioru punktów oraz stopnia wielomianu optymalnego, wyznaczy współczynniki wielomianu oraz błąd aproksymacji metodą najmniejszych kwadratów.

Można wziąć Gaussa z poprzedniego zadania jak trzeba dokładniejszy

```

def rozwiadz_uklad_gausa(A, b):
    n = len(b)

    for i in range(n):
        if A[i][i] == 0:
            raise ValueError("Na przekątnej pojawiło się zero - nie można wykonać eliminacji Gaussa.")

        for j in range(i + 1, n):
            wspolczynnik = A[j][i] / A[i][i]

            for k in range(i, n):
                A[j][k] = A[j][k] - wspolczynnik * A[i][k]

            b[j] = b[j] - wspolczynnik * b[i]

    x = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0
        for j in range(i + 1, n):
            suma += A[i][j] * x[j]

        x[i] = (b[i] - suma) / A[i][i]

    return x

def aproksymacja_najmniejszych_kwadratow(punkty, stopien):
    n = stopien + 1 # liczba współczynników

    A = [] # macierz układu
    B = [] # wyrazy wolne

    for i in range(n):
        wiersz = []

        for j in range(n):
            suma = 0.0

            for x, y in punkty:
                suma += x ** (i + j)

            wiersz.append(suma)

        A.append(wiersz)

        suma = 0.0

        for x, y in punkty:
            suma += y * (x ** i)

        B.append(suma)

```

```

A_kopia = []
for wiersz in A:
    A_kopia.append(wiersz.copy())

B_kopia = B.copy()

wspolczynniki = rozwiaz_uklad_gaussa(A_kopia, B_kopia)

h = 0.0 # błąd

for x, y in punkty:
    y_aprox = 0.0

    for i in range(len(wspolczynniki)):
        y_aprox += wspolczynniki[i] * (x ** i)

    h += (y_aprox - y) ** 2

return wspolczynniki, h

punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)]
stopien = 2

wspolczynniki, blad = aproksymacja_najmniejszych_kwadratow(punkty, stopien)

print("Współczynniki wielomianu:")
for i in range(len(wspolczynniki)):
    print(f"a{i} = {wspolczynniki[i]}")

print("\nSuma kwadratów błędów:")
print(blad)

```

Współczynniki wielomianu:

a0 = 1.98657142857143

a1 = 1.2337142857142795

a2 = -0.3828571428571398

Suma kwadratów błędów:

0.0017028571428571648

Zadanie 4

Rozwiązania dwóch pierwszych zadań przedstaw na wykresie, zaznaczając również zaobserwowane wartości funkcji.

```

import matplotlib.pyplot as plt

print("----- ZADANIE 4 -----")

# ===== Zadanie 1: aproksymacja liniowa =====
punkty1 = [(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)]

n1 = len(punkty1)
Sx = 0.0
Sy = 0.0
Sxx = 0.0
Sxy = 0.0

for x, y in punkty1:
    Sx += x
    Sy += y
    Sxx += x * x
    Sxy += x * y

a1 = (n1 * Sxy - Sx * Sy) / (n1 * Sxx - Sx * Sx)
b1 = (Sy - a1 * Sx) / n1

x1 = [p[0] for p in punkty1]
y1 = [p[1] for p in punkty1]

x1_wykres = []
y1_wykres = []

xmin1 = min(x1)
xmax1 = max(x1)

for i in range(200):
    X = xmin1 + (xmax1 - xmin1) * i / 199
    Y = a1 * X + b1
    x1_wykres.append(X)
    y1_wykres.append(Y)

plt.figure(figsize=(8, 5))
plt.scatter(x1, y1, label="Punkty wejściowe")
plt.plot(x1_wykres, y1_wykres, label="Aproksymacja liniowa")
plt.title("Zadanie 1 - aproksymacja liniowa")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# ===== Zadanie 2: aproksymacja wielomianem 2 stopnia =====
punkty2 = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)]

def rozwiaz_uklad_gaussa(A, b):

```



```

n = len(b)

for i in range(n):
    max_wiersz = i
    for j in range(i + 1, n):
        if abs(A[j][i]) > abs(A[max_wiersz][i]):
            max_wiersz = j

    A[i], A[max_wiersz] = A[max_wiersz], A[i]
    b[i], b[max_wiersz] = b[max_wiersz], b[i]

    for j in range(i + 1, n):
        wsp = A[j][i] / A[i][i]
        for k in range(i, n):
            A[j][k] -= wsp * A[i][k]
        b[j] -= wsp * b[i]

x = [0.0] * n
for i in range(n - 1, -1, -1):
    suma = 0.0
    for j in range(i + 1, n):
        suma += A[i][j] * x[j]
    x[i] = (b[i] - suma) / A[i][i]

return x

```

```

Sx = 0.0
Sx2 = 0.0
Sx3 = 0.0
Sx4 = 0.0
Sy = 0.0
Sxy = 0.0
Sx2y = 0.0

```

```

for x, y in punkty2:
    Sx += x
    Sx2 += x**2
    Sx3 += x**3
    Sx4 += x**4
    Sy += y
    Sxy += x * y
    Sx2y += x**2 * y

```

```

n2 = len(punkty2)

```

```

A = [
    [Sx4, Sx3, Sx2],
    [Sx3, Sx2, Sx],
    [Sx2, Sx, n2]
]

```

```

B = [Sx2y, Sxy, Sy]

```

```

a2, b2, c2 = rozwiaz_uklad_gaussa(A, B)

x2 = [p[0] for p in punkty2]
y2 = [p[1] for p in punkty2]

x2_wykres = []
y2_wykres = []

xmin2 = min(x2)
xmax2 = max(x2)

for i in range(200):
    X = xmin2 + (xmax2 - xmin2) * i / 199
    Y = a2 * X**2 + b2 * X + c2
    x2_wykres.append(X)
    y2_wykres.append(Y)

plt.figure(figsize=(8, 5))
plt.scatter(x2, y2, label="Punkty wejściowe")
plt.plot(x2_wykres, y2_wykres, label="Aproksymacja wielomianem 2 stopnia")
plt.title("Zadanie 2 - aproksymacja wielomianem 2 stopnia")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

```

Lab 10

Zadanie 1

Napisz program implementujący różniczkowanie numeryczne za pomocą metody Newtona dla następujących funkcji:

a) $f(x) = 2x^2 + 2$

b) $f(x) = 2x^4 - x^2 + 3x - 7$

c) $f(x) = x^2 e^x$

Oblicz błąd względny otrzymanego rozwiązania dla: $h = 10^{-2}$ oraz $h = 10^{-4}$.

```

def roznica_w_przod(f, x, h):
    return (f(x + h) - f(x)) / h

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona):
    if wartosc_dokladna != 0:
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
    else:
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna jest równa 0.")

def porownaj_pochodna(nazwa, f, df, x, h_lista):
    print("\n" + nazwa)
    print("x =", x)
    print("h          pochodna numeryczna          pochodna dokładna
błąd względny")

    for h in h_lista:
        pochodna_numeryczna = roznica_w_przod(f, x, h)
        pochodna_dokladna = df(x)
        blad = blad_wzgledny(pochodna_dokladna, pochodna_numeryczna)

        print(f"{h:<14} {pochodna_numeryczna:<25} {pochodna_dokladna:<25} {blad}")

x0 = 1.0

h_lista = [10**(-2), 10**(-4)]

```

punkt a)

```

def f1(x):
    return 2 * x**2 + 2

def df1(x):
    return 4 * x

porownaj_pochodna("a) f(x) = 2x^2 + 2", f1, df1, x0, h_lista)

```

```

a) f(x) = 2x^2 + 2
x = 1.0
h          pochodna numeryczna          pochodna dokładna          błąd względny
0.01      4.0200000000000046          4.0
0.0050000000000001144
0.0001    4.0001999999998344          4.0
4.999999958599233e-05

```

punkt b)

```
def f2(x):
    return 2 * x**4 - x**2 + 3*x - 7

def df2(x):
    return 8 * x**3 - 2 * x + 3
```

```
porownaj_pochodna("b) f(x) = 2x^4 - x^2 + 3x - 7", f2, df2, x0, h_lista)
```

b) $f(x) = 2x^4 - x^2 + 3x - 7$

$x = 1.0$

h	pochodna numeryczna	pochodna dokładna	błąd względny
0.01	9.110802000000007	9.0	
0.0123113333333334082			
0.0001	9.00110080006097	9.0	
0.00012223111178855106			

punkt c)

```
def f3(x):
    return x**2 * math.exp(x)

def df3(x):
    return math.exp(x) * (x**2 + 2 * x)
```

```
porownaj_pochodna("c) f(x) = x^2e^x", f3, df3, x0, h_lista)
```

c) $f(x) = x^2e^x$

$x = 1.0$

h	pochodna numeryczna	pochodna dokładna	błąd względny
0.01	8.250576695971112	8.154845485377136	
0.01173918141865927			
0.0001	8.155796942914684	8.154845485377136	
0.00011667388907050434			

Zadanie 2

Przeprowadź obliczenia analogiczne jak w zadaniu 1 dla metod różnic skończonych: wstecznej i centralnej dwupunktowej.

a) $f(x) = 2x^2 + 2$

b) $f(x) = 2x^4 - x^2 + 3x - 7$

c) $f(x) = x^2e^x$

Obliczenia wykonujemy dla: $h = 10^{-2}$ oraz: $h = 10^{-4}$.

```

def roznica_wsteczna(f, x, h):
    return (f(x) - f(x - h)) / h

def roznica_centralna_dwupunktowa(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona):
    if wartosc_dokladna != 0:
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
    else:
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna jest równa 0.")

def f1(x):
    return 2 * x**2 + 2

def df1(x):
    return 4 * x

def f2(x):
    return 2 * x**4 - x**2 + 3*x - 7

def df2(x):
    return 8 * x**3 - 2 * x + 3

def f3(x):
    return x**2 * math.exp(x)

def df3(x):
    return math.exp(x) * (x**2 + 2 * x)

def porownaj_metody(nazwa, f, df, x, h_lista):
    print("\n" + nazwa)
    print("x =", x)
    print("h          metoda          pochodna numeryczna          pochodna
dokładna          błąd względny")

    for h in h_lista:
        pochodna_dokladna = df(x)

        pochodna_wsteczna = roznica_wsteczna(f, x, h)
        blad_wsteczny = blad_wzgledny(pochodna_dokladna, pochodna_wsteczna)

        print(f"h:<14} {'wsteczna':<15} {pochodna_wsteczna:<25}
{pochodna_dokladna:<25} {blad_wsteczny}")

        pochodna_centralna = roznica_centralna_dwupunktowa(f, x, h)
        blad_centralny = blad_wzgledny(pochodna_dokladna, pochodna_centralna)

        print(f"h:<14} {'centralna':<15} {pochodna_centralna:<25}
{pochodna_dokladna:<25} {blad_centralny}")

```

```
x0 = 1.0
```

```
h_lista = [10**(-2), 10**(-4)]
```

```
porownaj_metody("a) f(x) = 2x^2 + 2", f1, df1, x0, h_lista)
```

```
porownaj_metody("b) f(x) = 2x^4 - x^2 + 3x - 7", f2, df2, x0, h_lista)
```

```
porownaj_metody("c) f(x) = x^2e^x", f3, df3, x0, h_lista)
```

a) $f(x) = 2x^2 + 2$

$x = 1.0$

h	metoda	pochodna numeryczna	pochodna dokładna
błąd względny			
0.01	wsteczna	3.9800000000000058	4.0
0.004999999999998561			
0.01	centralna	4.0000000000000026	4.0
6.439293542825908e-15			
0.0001	wsteczna	3.9998000000000775	4.0
4.999999806260575e-05			
0.0001	centralna	3.999999999995595	4.0
1.1013412404281553e-13			

b) $f(x) = 2x^4 - x^2 + 3x - 7$

$x = 1.0$

h	metoda	pochodna numeryczna	pochodna dokładna
błąd względny			
0.01	wsteczna	8.89079800000001	9.0
0.01213355555555435			
0.01	centralna	9.000800000000009	9.0
8.888888888888659e-05			
0.0001	wsteczna	8.998900080001704	9.0
0.00012221333314401918			
0.0001	centralna	9.0000000800039	9.0
8.889322265935739e-09			

c) $f(x) = x^2e^x$

$x = 1.0$

h	metoda	pochodna numeryczna	pochodna dokładna
błąd względny			
0.01	wsteczna	8.060292210953346	8.154845485377136
0.011594735251984607			
0.01	centralna	8.15543445346223	8.154845485377136
7.222308333733161e-05			
0.0001	wsteczna	8.153894145626062	8.154845485377136
0.00011665944532977143			
0.0001	centralna	8.154845544270373	8.154845485377136
7.2218703664531924e-09			

Zadanie 3

Przeprowadź obliczenia analogiczne jak w zadaniu 1. dla metod różnic skończonych: wprzód i wstecznej trzypunktowej oraz centralnej czteropunktowej.

a) $f(x) = 2x^2 + 2$

b) $f(x) = 2x^4 - x^2 + 3x - 7$

c) $f(x) = x^2 e^x$

Obliczenia wykonujemy dla: $h = 10^{-2}$ oraz: $h = 10^{-4}$.

```

def roznica_w_przod_trzypunktowa(f, x, h):
    return (-3 * f(x) + 4 * f(x + h) - f(x + 2 * h)) / (2 * h)

def roznica_wsteczna_trzypunktowa(f, x, h):
    return (3 * f(x) - 4 * f(x - h) + f(x - 2 * h)) / (2 * h)

def roznica_centralna_czteropunktowa(f, x, h):
    return (f(x - 2 * h) - 8 * f(x - h) + 8 * f(x + h) - f(x + 2 * h)) / (12 * h)

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona):
    if wartosc_dokladna != 0:
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
    else:
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna jest równa 0.")

def f1(x):
    return 2 * x**2 + 2

def df1(x):
    return 4 * x

def f2(x):
    return 2 * x**4 - x**2 + 3*x - 7

def df2(x):
    return 8 * x**3 - 2 * x + 3

def f3(x):
    return x**2 * math.exp(x)

def df3(x):
    return math.exp(x) * (x**2 + 2 * x)

def porownaj_metody(nazwa, f, df, x, h_lista):
    print("\n" + nazwa)
    print("x =", x)
    print("h          metoda          pochodna numeryczna
pochodna dokładna      błąd względny")

    for h in h_lista:
        pochodna_dokladna = df(x)

        pochodna_przod = roznica_w_przod_trzypunktowa(f, x, h)
        blad_przod = blad_wzgledny(pochodna_dokladna, pochodna_przod)

        print(f"{h:<14} {'w przód 3-punktowa':<25} {pochodna_przod:<25}
{pochodna_dokladna:<25} {blad_przod}")

        pochodna_wsteczna = roznica_wsteczna_trzypunktowa(f, x, h)
        blad_wsteczny = blad_wzgledny(pochodna_dokladna, pochodna_wsteczna)

```



```
print(f"{h:<14} {'wsteczna 3-punktowa':<25} {pochodna_wsteczna:<25}\n{pochodna_dokladna:<25} {blad_wsteczny}")

pochodna_centralna = roznica_centralna_czteropunktowa(f, x, h)
blad_centralny = blad_wzgledny(pochodna_dokladna, pochodna_centralna)

print(f"{h:<14} {'centralna 4-punktowa':<25} {pochodna_centralna:<25}\n{pochodna_dokladna:<25} {blad_centralny}")

x0 = 1.0

h_lista = [10**(-2), 10**(-4)]

porownaj_metody("a)  $f(x) = 2x^2 + 2$ ", f1, df1, x0, h_lista)
porownaj_metody("b)  $f(x) = 2x^4 - x^2 + 3x - 7$ ", f2, df2, x0, h_lista)
porownaj_metody("c)  $f(x) = x^2e^x$ ", f3, df3, x0, h_lista)
```

$$a) f(x) = 2x^2 + 2$$

$$x = 1.0$$

h	metoda	pochodna numeryczna	pochodna
dokładna	błąd względny		
0.01	w przód 3-punktowa	4.000000000000092	4.0
	2.3092638912203256e-14		
0.01	wsteczna 3-punktowa	4.000000000000036	4.0
	8.881784197001252e-16		
0.01	centralna 4-punktowa	4.000000000000034	4.0
	8.43769498715119e-15		
0.0001	w przód 3-punktowa	3.999999999995595	4.0
	1.1013412404281553e-13		
0.0001	wsteczna 3-punktowa	4.00000000000178	4.0
	4.4497738826976274e-13		
0.0001	centralna 4-punktowa	4.000000000001039	4.0
	2.5979218776228663e-13		

$$b) f(x) = 2x^4 - x^2 + 3x - 7$$

$$x = 1.0$$

h	metoda	pochodna numeryczna	pochodna
dokładna	błąd względny		
0.01	w przód 3-punktowa	8.99838800000013	9.0
	0.00017911111110969892		
0.01	wsteczna 3-punktowa	8.998412	9.0
	0.0001764444444443586		
0.01	centralna 4-punktowa	9.00000000000016	9.0
	1.7763568394002505e-15		
0.0001	w przód 3-punktowa	8.999999839995887	9.0
	1.777823478556052e-08		
0.0001	wsteczna 3-punktowa	8.999999840020312	9.0
	1.777552090705588e-08		
0.0001	centralna 4-punktowa	9.00000000000493	9.0
	5.477100254817439e-13		

$$c) f(x) = x^2 e^x$$

$$x = 1.0$$

h	metoda	pochodna numeryczna	pochodna
dokładna	błąd względny		
0.01	w przód 3-punktowa	8.1536531934717	
8.154845485377136	0.0001462065599617194		
0.01	wsteczna 3-punktowa	8.15368173640505	
8.154845485377136	0.0001427064405049649		
0.01	centralna 4-punktowa	8.154845457287603	
8.154845485377136	3.4445205035658563e-09		
0.0001	w przód 3-punktowa	8.154845367567276	
8.154845485377136	1.4446608443622994e-08		
0.0001	wsteczna 3-punktowa	8.154845367591701	
8.154845485377136	1.4443613303299912e-08		
0.0001	centralna 4-punktowa	8.15484548537378	
8.154845485377136	4.11477823294636e-13		

Zadanie 4

Zaimplementuj różniczkowanie za pomocą wielomianów Lagrange'a. Wyznacz pochodną w punkcie $x = 3.5$ przy następujących węzłach interpolacji: $\{(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)\}$.

Interpolacja Lagrange'a

```
print("----- ZADANIE 4 -----")

def baza_lagrange(punkty, i, x):
    xi = punkty[i][0]

    wynik = 1.0

    for j in range(len(punkty)):
        if j != i:
            xj = punkty[j][0]
            wynik *= (x - xj) / (xi - xj)

    return wynik

def wielomian_lagrange(punkty, x):
    suma = 0.0

    for i in range(len(punkty)):
        yi = punkty[i][1]
        suma += yi * baza_lagrange(punkty, i, x)

    return suma

def pochodna_lagrange_centralna(punkty, x, h):
    return (wielomian_lagrange(punkty, x + h) - wielomian_lagrange(punkty, x - h)) / (2 * h)

punkty = [(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)]

X = 3.5
h = 10**(-4)

wynik = pochodna_lagrange_centralna(punkty, X, h)

print("Węzły interpolacji:")
print(punkty)

print("\nPunkt, w którym liczymy pochodną:")
print("x =", X)

print("\nKrok:")
print("h =", h)

print("\nPrzybliżona wartość pochodnej:")
print("f'(", X, ") =", wynik)
```

Węzły interpolacji:

$[(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)]$

Punkt, w którym liczymy pochodną:

$x = 3.5$

Krok:

$h = 0.0001$

Przybliżona wartość pochodnej:

$f'(3.5) = 14.000000000020663$

uniwersalna dla różniczkowania wielomianem Lagrange'a

```

def pochodna_bazy_lagrange(punkty, i, x):
    xi = punkty[i][0]

    suma = 0.0

    # Liczymy pochodną i-tej funkcji bazowej Lagrange'a
    for m in range(len(punkty)):
        if m != i:
            xm = punkty[m][0]

            iloczyn = 1.0

            for j in range(len(punkty)):
                if j != i and j != m:
                    xj = punkty[j][0]
                    iloczyn *= (x - xj) / (xi - xj)

            iloczyn = iloczyn / (xi - xm)

            suma += iloczyn

    return suma


def pochodna_lagrange(punkty, x):
    suma = 0.0

    #  $f'(x) \approx \sum y_i * l_i'(x)$ 
    for i in range(len(punkty)):
        yi = punkty[i][1]
        suma += yi * pochodna_bazy_lagrange(punkty, i, x)

    return suma


def sprawdz_punkty(punkty):
    if len(punkty) < 2:
        raise ValueError("Potrzeba co najmniej dwóch punktów.")

    for i in range(len(punkty)):
        for j in range(i + 1, len(punkty)):
            if punkty[i][0] == punkty[j][0]:
                raise ValueError("Wartości x w punktach nie mogą się powtarzać.")


def pochodna_lagrange_bezpiecznie(punkty, x):
    sprawdz_punkty(punkty)
    return pochodna_lagrange(punkty, x)


punkty = [
    (1, 4),

```

```

(2, 10),
(3, 20),
(4, 34),
(5, 52)
]

X = 3.5

wynik = pochodna_lagrange_bezpiecznie(punkty, X)

print("Węzły interpolacji:")
print(punkty)

print("\nPunkt, w którym liczymy pochodną:")
print("x =", X)

print("\nPrzybliżona wartość pochodnej:")
print("f'(", X, ") =", wynik)

```

Lab 11

Zadanie 1

Zaimplementuj całkowanie numeryczne za pomocą metody prostokątów.

Całość to:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f(x_i + \frac{h}{2})$$

PYTHON

```

def metoda_prostokatow(f, a, b, n):
    h = (b - a) / n # podstawa prostokąta
    # f(x_srodek) -> wysokość prostokąta

    suma = 0.0

    for i in range(n):
        x_i = a + i * h # początek przedziału
        x_srodek = x_i + h / 2 # środek przedziału
        # x_srodek = a + (i + 0.5) * h # to samo co 2 linijki wyżej

        suma += f(x_srodek) # dodawanie wysokości wszystkich prostokątów

    return h * suma

```

```
def f1(x):
    return x**2

wynik_prostokaty = metoda_prostokatow(f1, 0, 1, 100)

print("\nMetoda prostokątów:")
print("Wynik =", wynik_prostokaty)
```

Zadanie 2

Zaimplementuj całkowanie numeryczne za pomocą metody trapezów.

```
def metoda_trapezow(f, a, b, n):
    h = (b - a) / n

    suma = (f(a) + f(b)) / 2

    for i in range(1, n):
        x = a + i * h
        suma += f(x)

    return h * suma
```

```
def f1(x):
    return x**2

wynik_trapezy = metoda_trapezow(f1, 0, 1, 100)

print("\nMetoda trapezów:")
print("Wynik =", wynik_trapezy)
```

dokładnie ze wzorem:

$$\int_a^b f(x)dx = h \left[\frac{1}{2}(y_0 + y_n) + \sum_{i=1}^{n-1} y_i \right]$$


```
def metoda_trapezow(f, a, b, n):
    h = (b - a) / n

    lewa_nawiasu = (1/2) * (f(a) + f(b))
    suma = 0.0

    for i in range(1, n):
        x_i = a + i * h
        suma += f(x_i)

    nawias = lewa_nawiasu + suma

    return h * nawias
```

Zadanie 3

Zaimplementuj całkowanie numeryczne za pomocą metody Simpsona.

```
def metoda_simpsona(f, a, b, n):
    if n % 2 != 0:
        raise ValueError("W metodzie Simpsona liczba podprzedziałów n musi być parzysta.")

    h = (b - a) / n

    suma = f(a) + f(b)

    for i in range(1, n):
        x_i = a + i * h

        if i % 2 == 0:
            suma += 2 * f(x_i)
        else:
            suma += 4 * f(x_i)

    return (h / 3) * suma
```

```
def f1(x):
    return x**2

wynik_simpson = metoda_simpsona(f1, 0, 1, 100)

print("\nMetoda Simpsona:")
print("Wynik =", wynik_simpson)
```

Zadanie 4

Wyniki powyższych programów przetestuj dla następujących całek:

a)

$$\int_0^1 x^2 dx,$$

b)

$$\int_0^{1/2} \cos x dx,$$

c)

$$\int_e^{e^2} \frac{1}{x} dx.$$

Zadanie 5

Sprawdź dokładność otrzymanych rozwiązań.

Cały kod do zadań z lab 11

PYTHON

```
import math

# ===== ZADANIE 1 =====

def metoda_prostokatow(f, a, b, n):
    h = (b - a) / n # podstawa prostokąta
    # f(x_srodek) -> wysokość prostokąta

    suma = 0.0

    for i in range(n):
        x_i = a + i * h # początek przedziału
        x_srodek = x_i + h / 2 # środek przedziału
        # x_srodek = a + (i + 0.5) * h # to samo co 2 linijki wyżej

        suma += f(x_srodek) # dodawanie wysokości wszystkich prostokątów

    return h * suma

# ===== ZADANIE 2 =====

def metoda_trapezow(f, a, b, n):
    h = (b - a) / n

    suma = (f(a) + f(b)) / 2

    for i in range(1, n):
        x = a + i * h
        suma += f(x)

    return h * suma

# ===== ZADANIE 3 =====

def metoda_simpsona(f, a, b, n):
    if n % 2 != 0:
        raise ValueError("W metodzie Simpsona liczba podprzedziałów n musi być parzysta.")

    h = (b - a) / n

    suma = f(a) + f(b)

    for i in range(1, n):
        x = a + i * h

        if i % 2 == 0:
            suma += 2 * f(x)
        else:
```

```

        suma += 4 * f(x)

    return (h / 3) * suma

# ===== ZADANIE 5 =====

def blad_bezwzględny(wartosc_dokladna, wartosc_przyblizona):
    return abs(wartosc_dokladna - wartosc_przyblizona)

def blad_względny(wartosc_dokladna, wartosc_przyblizona):
    if wartosc_dokladna == 0:
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna jest równa 0.")

    return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)

# ===== FUNKCJE Z ZADANIA 4 =====

def f1(x):
    return x**2

def f2(x):
    return math.cos(x)

def f3(x):
    return 1 / x

# ===== TESTOWANIE METOD =====

def testuj_calke(nazwa, f, a, b, wartosc_dokladna, n):
    print("\n" + nazwa)
    print("Przedział całkowania:", "[", a, ",", b, "]")
    print("Liczba podprzedziałów n =", n)
    print("Wartość dokładna =", wartosc_dokladna)

    wynik_prostokaty = metoda_prostokatow(f, a, b, n)
    wynik_trapezy = metoda_trapezow(f, a, b, n)
    wynik_simpson = metoda_simpsona(f, a, b, n)

    print("\nMetoda prostokątów:")
    print("Wynik =", wynik_prostokaty)
    print("Błąd bezwzględny =", blad_bezwzględny(wartosc_dokladna,
    wynik_prostokaty))
    print("Błąd względny =", blad_względny(wartosc_dokladna, wynik_prostokaty))

    print("\nMetoda trapezów:")
    print("Wynik =", wynik_trapezy)

```

```

print("Błąd bezwzględny =", blad_bezwzgledny(wartosc_dokladna, wynik_trapezy))
print("Błąd względny =", blad_wzgledny(wartosc_dokladna, wynik_trapezy))

print("\nMetoda Simpsona:")
print("Wynik =", wynik_simpson)
print("Błąd bezwzględny =", blad_bezwzgledny(wartosc_dokladna, wynik_simpson))
print("Błąd względny =", blad_wzgledny(wartosc_dokladna, wynik_simpson))

# ===== ZADANIE 4 =====

n = 100

testuj_calke(
    "a) całka od 0 do 1 z x^2 dx",
    f1,
    0,
    1,
    1 / 3,
    n
)

testuj_calke(
    "b) całka od 0 do pi/2 z cos(x) dx",
    f2,
    0,
    math.pi / 2,
    1,
    n
)

testuj_calke(
    "c) całka od e do e^2 z 1/x dx",
    f3,
    math.e,
    math.e**2,
    1,
    n
)

```

Lab 12

Zadanie 1